

MI VDEC API

Version 3.04

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.00	Initial release	12/04/2020
3.01	新增 MI_VDEC_PauseChn, MI_VDEC_RefreshChn 和 MI_VDEC_ResumeChn, 用于暂停解码后刷重复帧功能	01/04/2021
3.02	新增 MI_VDEC_StepChn, 用于暂停解码后单帧播放功能	02/24/2021
3.03	- 新增 Muffin 芯片解码规格 - 新增 Muffin 数据流程图 - 新增 API - MI_VDEC_SetOutputPortAttrEx - MI_VDEC_GetOutputPortAttrEx - MI_VDEC_SetDestCropEx - MI_VDEC_GetDestCropEx	05/19/2021
3.04	- 删除 MI_VDEC_CreateDev 原参数 bDisableLowLatency, 在 MI_VDEC_CreateChn 新增该参数, 且支持按 chn 设置 - MI_VDEC_CreateDev 新增参数解码最大宽和高, 用于调整解码器支持的最大码流分辨率 - MI_VDEC_GetChnStat 新增错误类型	07/20/2021
	新增 PROCFS 介绍	08/25/2021

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. 概述.....	1
1.1. 模块简介.....	1
1.2. 解码流程图.....	2
1.2.1 Taiyaki 数据流程图.....	2
1.2.2 Takoyaki 数据流程图.....	2
1.2.3 Tiramisu 数据流程图.....	2
1.2.4 Muffin 数据流程图.....	2
1.2.5 Mochi 数据流程图.....	3
1.3. 重要概念.....	3
2. API 参考.....	6
2.1. MI_VDEC_CreateDev.....	7
2.2. MI_VDEC_DestroyDev.....	9
2.3. MI_VDEC_CreateChn.....	10
2.4. MI_VDEC_DestroyChn.....	13
2.5. MI_VDEC_StartChn.....	14
2.6. MI_VDEC_StopChn.....	15
2.7. MI_VDEC_GetChnAttr.....	16
2.8. MI_VDEC_GetChnStat.....	17
2.9. MI_VDEC_FlushChn.....	18
2.10. MI_VDEC_ResetChn.....	19
2.11. MI_VDEC_SetChnParam.....	20
2.12. MI_VDEC_GetChnParam.....	21
2.13. MI_VDEC_SendStream.....	22
2.14. MI_VDEC_PauseChn.....	24
2.15. MI_VDEC_RefreshChn.....	26
2.16. MI_VDEC_ResumeChn.....	30
2.17. MI_VDEC_StepChn.....	31
2.18. MI_VDEC_GetUserData.....	34
2.19. MI_VDEC_ReleaseUserData.....	35
2.20. MI_VDEC_SetDisplayMode.....	36
2.21. MI_VDEC_GetDisplayMode.....	37
2.22. MI_VDEC_SetOutputPortAttr.....	38
2.23. MI_VDEC_GetOutputPortAttr.....	39
2.24. MI_VDEC_SetOutputPortLayoutMode.....	40
2.25. MI_VDEC_GetOutputPortLayoutMode.....	41
2.26. MI_VDEC_SetDestCrop.....	42
2.27. MI_VDEC_GetDestCrop.....	43
2.28. MI_VDEC_SetChnErrHandlePolicy.....	44
3. 数据类型.....	46
3.1. MI_VDEC_CodecType_e.....	46
3.2. MI_VDEC_DPB_BufMode_e.....	47

3.3.	MI_VDEC_JpegFormat_e.....	48
3.4.	MI_VDEC_VideoMode_e.....	48
3.5.	MI_VDEC_ErrCode_e.....	49
3.6.	MI_VDEC_Decomode_e.....	50
3.7.	MI_VDEC_OutputOrder_e.....	50
3.8.	MI_VDEC_VideoFormat_e.....	51
3.9.	MI_VDEC_DisplayMode_e.....	52
3.10.	MI_VDEC_OutbufLayoutMode_e.....	52
3.11.	MI_VDEC_InitParam_t.....	53
3.12.	MI_VDEC_ChAttr_t.....	54
3.13.	MI_VDEC_JpegAttr_t.....	55
3.14.	MI_VDEC_VideoAttr_t.....	56
3.15.	MI_VDEC_ChnStat_t.....	57
3.16.	MI_VDEC_ChnParam_t.....	58
3.17.	MI_VDEC_VideoStream_t.....	58
3.18.	MI_VDEC_UserData_t.....	59
3.19.	MI_VDEC_OutputPortAttr_t.....	60
3.20.	MI_VDEC_ErrHandlePolicy_t.....	60
3.21.	MI_VDEC_CropCfg_t.....	61
4.	错误码.....	62
5.	PROCFS 介绍.....	63
5.1.	cat.....	63
5.2.	echo.....	67

1. 概述

1.1. 模块简介

视频解码功能参考，提供解码通道创建、码流传送及控制、输出图像裁剪和缩放等功能。

VDEC 模块的输入源主要是两类：

- 用户读取码流文件向解码模块发送数据；
- 用户将网络接收到的码流数据直接发送到解码模块。

表 1-1芯片解码规格

芯片	硬件解码模块	支持最大通道数	支持协议	分辨率范围
Taiyaki	DEC2.0	16	H.264/H.265	H.264: max 4096x4096, min 176x128 H.265: max 4096x4096, min 176x128
Takoyaki	DEC2.0	4	H.264/H.265	H.264: max 4096x4096, min 176x128 H.265: max 4096x4096, min 176x128
Tiramisu	DEC2.0	16	H.264/H.265	H.264: max 4096x4096, min 176x128 H.265: max 4096x4096, min 176x128
Mochi	DEC2.0	64x2	H.264/H.265	H.264: max 4096x4096, min 176x128 H.265: max 4096x4096, min 176x128
Muffin	DEC2.0	64x2	H.264/H.265	H.264: max 4096x4096, min 176x128 H.265: max 4096x4096, min 176x128

1.2. 解码流程图

1.2.1 Taiyaki 数据流程图

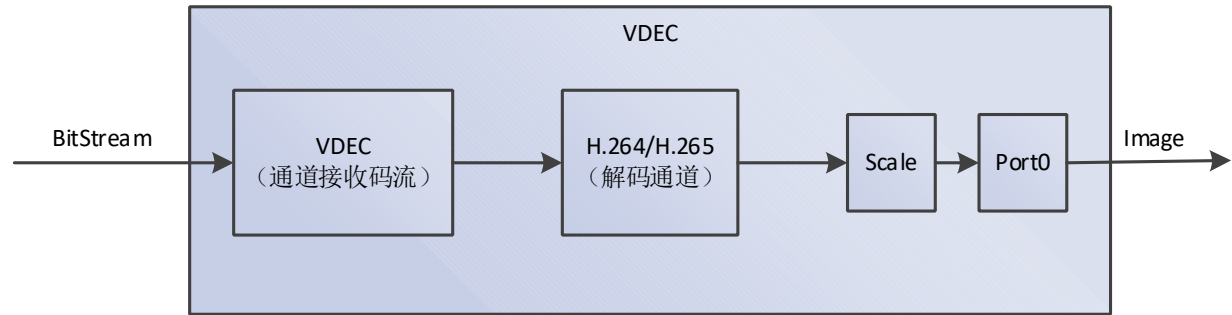


图 1-1 VDEC 数据流程图

1.2.2 Takoyaki 数据流程图

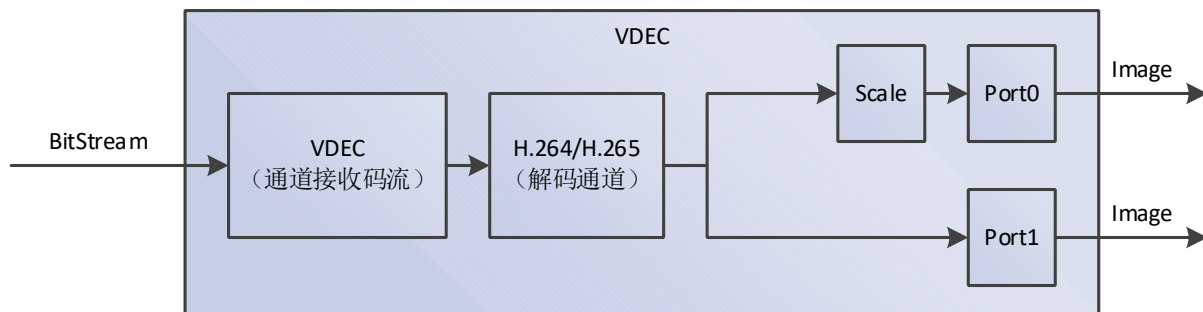


图 1-2 VDEC 数据流程图

1.2.3 Tiramisu 数据流程图

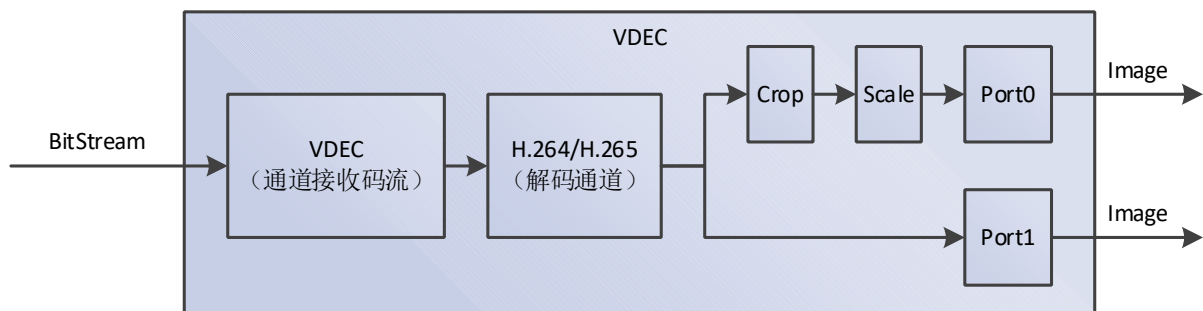


图 1-3 VDEC 数据流程图

1.2.4 Muffin 数据流程图

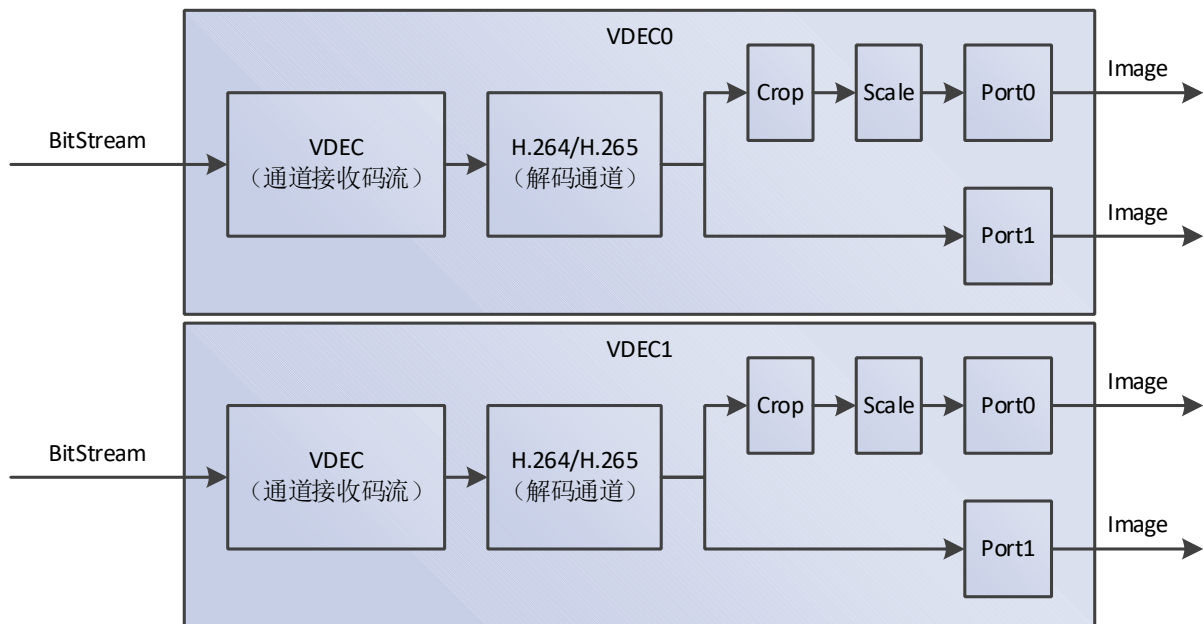


图 1-4 VDEC 数据流程图

Muffin 总共有两个 VDEC 硬件，两硬件解码能力一致。通过 Device ID 区分，分别为 Device0 与 Device1。

1.2.5 Mochi 数据流程图

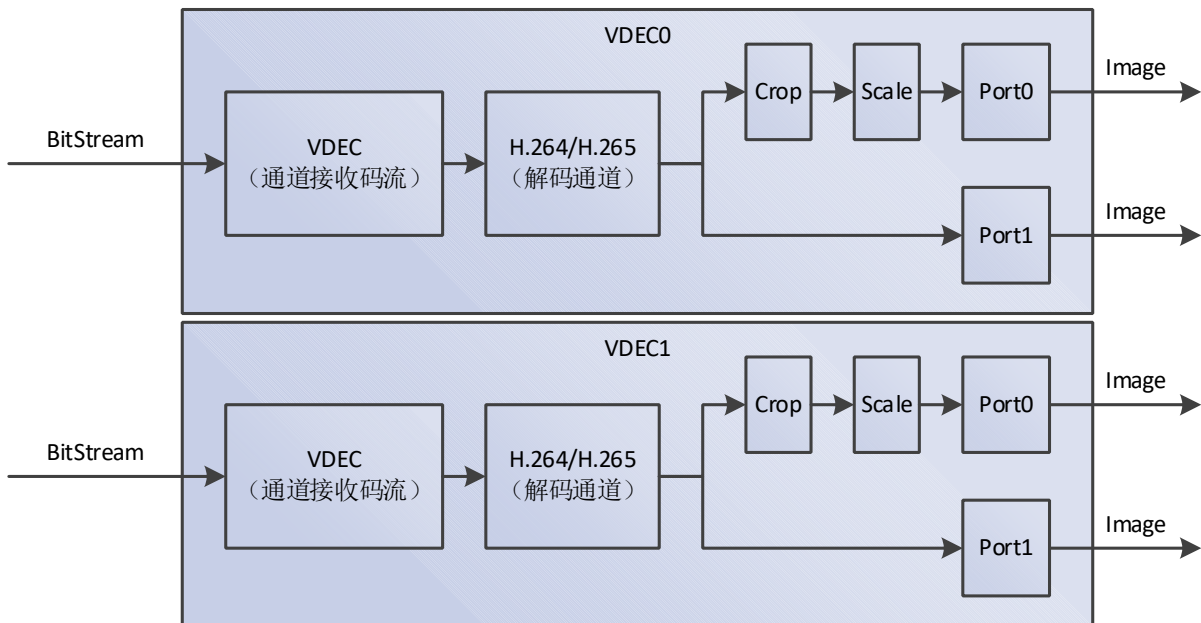


图 1-5 VDEC 数据流程图

Mochi 总共有两个 VDEC 硬件，两硬件解码能力一致。通过 Device ID 区分，分别为 Device0 与 Device1。

1.3. 重要概念

- 码流发送方式

VDEC 解码器提供两种码流发送方式：

- 按流发送（E_MI_VDEC_VIDEO_MODE_STREAM）：用户每次可发送任意长度码流到解码器，由解码器内部完成码流帧的解析和拼接过程。对于 H.264/H.265，需在收到下一帧码流才能确认为当前码流帧的结束，所以在该模式下，输入一帧 H.264/H.265 码流，不会立刻解出图像。目前不支持按流发送功能。
- 按帧发送（E_MI_VDEC_VIDEO_MODE_FRAME）：用户每次发送完整的一帧码流到解码器，每调用一次发送接口，解码器就认为该帧码流已经结束，并开始解码图像。因此，用户需保证每次发送的码流必须为完整的一帧，否则会出现解码错误。使用该模式可以达到快速解码的目的。

码流发送方式 eVideoMode 可在接口 MI_VDEC_CreateChn 中设置。

- 图像输出顺序

根据 H.264/H.265 协议，解码图像可能不会在解码后立即输出。VDEC 解码器可以通过设置不同的图像输出方式达到尽快输出的目的。图像输出方式包括以下两种：

- 解码序：解码图像按照解码的先后顺序输出。
- 显示序：解码图像按照协议中的显示顺序输出。

根据 H.264/H.265 协议，视频的解码顺序（即解码序）和解码图像的显示顺序（即显示序）可能不一致。例如：解码 B 帧时，需要前后的 P 帧作为参考，所以 B 帧后的 P 帧先于 B 帧解码，但 B 帧先于 P 帧输出。按解码序输出是保证快速输出的一个必要条件，用户选择按解码序输出，需保证码流的解码序与显示序相同。

按帧发送码流与按解码序输出相结合能达到快速解码和快速输出的目的，用户必须保证每次发送的是完整的一帧码流以及码流的解码序和显示序相同。

目前图像输出方式会自适应为显示序，用户设置无效。

- 时间戳（PTS）处理

在按帧模式发送码流时，解码输出的图像时间戳 PTS 为发送码流接口(MI_VDEC_SendStream)中用户送

入的 PTS，解码器不会更改此值；如果用户送入的 PTS 值为-2，则表示此图像不会被视频显示模块（DISP）显示；如果是其他值，则表示解码器不做任何处理。

该 PTS 不用于帧率控制。帧率控制功能请参考 mi_sys 模块的 api 说明。

- 码流 Buffer

码流 Buffer 用于缓存用户输入的码流，然后送入解码器解码。码流 buffer 的大小没有固定的计算公式，一般按照经验值，分辨率小于或等于 D1,建议配置为 512KB；分辨率在(D1,1080P],建议配置为 1MB；分辨率在(1080P, 4K], 建议配置为 2MB；用户可根据实际需求，选择合适的值设置。

- eDpbBufMode

eDpbBufMode 可供用户设置不同的 buffer 模式进行解码。根据不同的场景，选择不同的 buffer 模式，可以达到省内存的目的。当设为 E_MI_VDEC_DPB_MODE_NORMAL，则不能节省内存；当设为 E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF，则用于解码仅包含一个参考帧的码流，此时仅需一个 DPB buffer 即可；当设为 E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF，则用于解码仅包含两个参考帧的码流，此时仅需两个 DPB buffer 即可。

- u32RefFrameNum

u32RefFrameNum 表示最大参考帧个数。由于在实际的场景中，系统的内存并不是无限大，用户可以根据产品的定义对参考帧的个数进行限制，以便提示用户当前码流可能超规格。如果用户设置了 eDpbBufMode 为 E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF 或者 E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF，则该参数无效。如果用户设置了 eDpbBufMode 为 E_MI_VDEC_DPB_MODE_NORMAL，则解码器在参考帧个数受到限制时进行解码，解码图像可能会异常或者解码超时。

- 输出图像缩放

用户调用 MI_VDEC_SetOutputPortAttr 可以对解码后的图像进行缩放，以输出需要的分辨率的图像。但需注意，解码器仅支持对图像缩小，不支持对图像放大。

- 低延时

这里的延时是指解码 B 帧后，需要对待显示的图像队列进行重排序（reorder）所产生的延时。当解码不含 B 帧的码流时，解码顺序和显示顺序相同，因此不会出现延时输出，即低延时；而解码含 B 帧的码流时，由于 B 帧有前后向参考，需要等待所参考的图像解码完后，才能解当前帧，并且需要对解码后的图像重排序（reorder），由此产生输出延时。这两种模式需要通过设置 bDisableLowLatency 来完成。默认 bDisableLowLatency 为 FALSE，用户无需设置；当需要解码 B 帧码流时，需要将 bDisableLowLatency 设为 TRUE。

- 输出 buffer 模式

用户可以调用 MI_VDEC_SetOutputPortLayoutMode 设置输出 buffer 的模式。该接口可使用于芯片 Takoyaki, Tiramisu, Muffin。当用户需要手动控制输出 buffer 的模式时，可调用该接口。当为 E_MI_VDEC_OUTBUF_LAYOUT_LINEAR 模式时，表示输出 buffer 为线性模式；当为 E_MI_VDEC_OUTBUF_LAYOUT_TILE 模式时，表示输出 buffer 为 TILE 模式；默认为 E_MI_VDEC_OUTBUF_LAYOUT_AUTO 模式，表示输出 buffer 会自动在线性模式和 TILE 模式间切换。

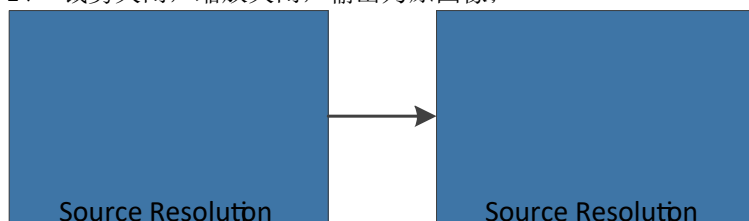
- 输出图像裁剪

用户调用 MI_VDEC_SetDestCrop 可以对解码后的图像进行裁剪再输出。设置裁剪时，坐标 X 和宽必须按 16 对齐，坐标 Y 和高必须按 2 对齐，否则接口会返回错误码 MI_ERR_VDEC_ILLEGAL_PARAM。如果裁剪功能和缩放功能同时使用，则按照先裁剪后缩放的处理顺序，裁剪的图像大小不能小于缩放的大小，否则接口会返回错误码 MI_ERR_VDEC_ILLEGAL_PARAM。

Taiyaki 和 Takoyaki 系列芯片不支持裁剪功能。

裁剪和缩放的关系：

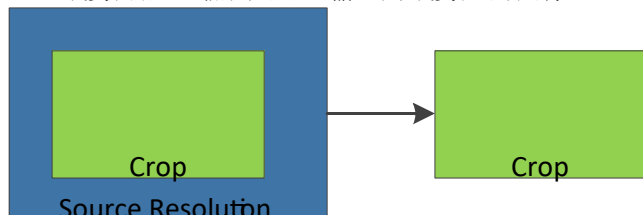
1、 裁剪关闭，缩放关闭，输出为原图像：



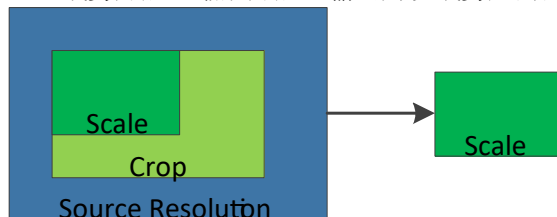
2、 裁剪关闭，缩放开启，输出为缩放后的图像：



3、 裁剪开启，缩放关闭，输出为裁剪后的图像：



4、 裁剪开启，缩放开启，输出为以裁剪后的的图像为参考，再做缩放后的图像：



如果开启了缩放，则最后输出的 **frame buffer** 以缩放大小为最终输出大小。

5、 输出图像格式

目前 H264 及 H265 解码只支持输出 NV12 图像格式。

2. API 参考

该功能模块提供以下 API:

API名	功能
MI_VDEC_CreateDev	创建解码设备
MI_VDEC_DestroyDev	销毁解码设备
MI_VDEC_CreateChn	创建解码通道
MI_VDEC_DestroyChn	销毁解码通道
MI_VDEC_GetChnAttr	获取解码通道属性
MI_VDEC_StartChn	开始通道解码
MI_VDEC_StopChn	停止通道解码
MI_VDEC_GetChnStat	获取解码通道状态
MI_VDEC_FlushChn	清理解码通道缓存数据
MI_VDEC_ResetChn	复位解码通道
MI_VDEC_SetChnParam	设置解码通道参数
MI_VDEC_GetChnParam	获取解码通道参数
MI_VDEC_SendStream	向解码通道发送码流数据
MI_VDEC_PauseChn	暂停通道解码
MI_VDEC_RefreshChn	刷新通道，再次解码当前帧
MI_VDEC_ResumeChn	恢复通道解码
MI_VDEC_StepChn	通道单帧解码。
MI_VDEC_GetUserData	获取解码通道的用户数据
MI_VDEC_ReleaseUserData	释放解码通道的用户数据
MI_VDEC_SetDisplayMode	设置解码通道显示模式
MI_VDEC_GetDisplayMode	获取解码通道显示模式
MI_VDEC_SetOutputPortAttr	设置解码通道输出端口属性
MI_VDEC_GetOutputPortAttr	获取解码通道输出端口属性
MI_VDEC_SetOutputPortLayoutMode	设置输出端口Buffer模式
MI_VDEC_GetOutputPortLayoutMode	获取输出端口Buffer模式
MI_VDEC_SetDestCrop	设置解码通道裁剪属性
MI_VDEC_GetDestCrop	获取解码通道裁剪属性

API名	功能
MI_VDEC_SetChnErrHandlePolicy	设置解码通道错误宏块的数据帧的输出策略

2.1. MI_VDEC_CreateDev

➤ 功能

创建解码设备。

➤ 定义

MI_S32 MI_VDEC_CreateDev(MI_VDEC_DEV VdecDev, [MI_VDEC_InitParam_t](#) *pstInitParam);

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
pstInitParam	解码设备初始化参数指针，数据类型： MI_VDEC_InitParam_t 。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_FAILED	失败。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INITED	设备已经初始化。
MI_ERR_VDEC_NULL_PTR	输入参数为空指针。
MI_ERR_VDEC_NOT_CONFIG	使用前未配置。
MI_ERR_VDEC_ILLEGAL_PARAM	非法参数或输入参数超过通道解码能力。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 设备号不能超出最大的设备号范围。
- 该接口在不同平台的使用上有所不同，具体见以下表格。

芯片/平台	接口使用描述
Taiyaki	无此接口，不支持设置。
Takoyaki	可选调用。
Tiramisu	可选调用。
Muffin	必须调用。

- 若选择使用该接口，则必须在创建解码通道前调用，否则会返回设备已初始化的错误码 MI_ERR_VDEC_INITED。
- 若重复调用该接口，则会返回设备已初始化的错误码 MI_ERR_VDEC_INITED。

➤ 举例

```
MI_S32 StartVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
    MI_VDEC_CHN VdecChn = 0;
    MI_VDEC_InitParam_t stInitParam;
    MI_VDEC_ChnAttr_t stChnAttr;

    memset(&stInitParam, 0x0, sizeof(MI_VDEC_InitParam_t));
    memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));

    s32Ret = MI_VDEC_CreateDev(VdecDev, &stInitParam);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_CreateDev failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    VdecChn = 0;
    stChnAttr.eCodecType = E_MI_VDEC_CODEC_TYPE_H264;
    stChnAttr.u32PicWidth = 1920;
    stChnAttr.u32PicHeight = 1080;
    stChnAttr.eVideoMode = E_MI_VDEC_VIDEO_MODE_FRAME;
    stChnAttr.u32BufSize = 1024*1024;
    stChnAttr.eDpbBufMode = E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF;
    stChnAttr.stVdecVideoAttr.u32RefFrameNum = 1;
    stChnAttr.u32Priority = 0;
    stChnAttr.stVdecVideoAttr.stErrHandlePolicy.bUseCusPolicy = FALSE;
    stChnAttr.stVdecVideoAttr.bDisableLowLatency = FALSE;

    s32Ret = MI_VDEC_CreateChn(VdecDev, VdecChn, &stChnAttr);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_CreateChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    return MI_SUCCESS;
}
```

➤ 相关主题

无

2.2. MI_VDEC_DestroyDev

➤ 功能

销毁解码设备。

➤ 定义

MI_S32 MI_VDEC_DestroyDev (MI_VDEC_DEV VdecDev);

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_FAILED	失败。
MI_ERR_VDEC_INVALID_DEVID	设备ID超出合法范围。
MI_ERR_VDEC_NOT_INIT	设备未初始化。
MI_ERR_VDEC_NOT_CONFIG	使用前未配置。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 该接口的使用因平台的不同而有所差异，具体请参照[MI_VDEC_CreateDev](#)接口的描述；
- 设备号不能超出最大的设备号范围；
- 若选择调用该接口，则必须在销毁所有解码通道后调用；如果用户未主动销毁所有解码通道，则该接口内部会做销毁所有解码通道的处理；
- 若重复调用该接口，则会返回设备未初始化的错误码MI_ERR_VDEC_NOT_INIT。

➤ 举例

```
MI_S32 StopVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
    MI_VDEC_CHN VdecChn = 0;

    s32Ret = MI_VDEC_DestroyChn(VdecDev, VdecChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_DestroyChn failed, s32Ret: 0x%x.\n", s32Ret);
    }
}
```

```

    return s32Ret;
}

//Confirm that you have destroyed all channels
s32Ret = MI_VDEC_DestroyDev(VdecDev);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_DestroyDev failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

return MI_SUCCESS;
}

```

- 相关主题
无

2.3. MI_VDEC_CreateChn

- 功能

创建视频解码通道。

- 定义

MI_S32 MI_VDEC_CreateChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
[MI_VDEC_ChnAttr_t](#) *pstChnAttr);

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围: [0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	解码通道号。 取值范围: [0, MI_VDEC_MAX_CHN_NUM)。	输入
pstChnAttr	解码通道属性指针, 数据类型: MI_VDEC_ChnAttr_t 。	输入

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_FAILED	失败。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围, 取值范围: [0, MI_VDEC_MAX_CHN_NUM)。
MI_ERR_VDEC_NOT_INIT	模块没有成功加载。
MI_ERR_VDEC_ILLEGAL_PARAM	非法参数或输入参数超过通道解码能力。

返回值	描述
MI_ERR_VDEC_CHN_EXIST	试图创建已经存在的通道。
MI_ERR_VDEC_NOMEM	分配内存失败（如系统内存不足）。
MI_ERR_VDEC_NULL_PTR	输入参数为空指针。
MI_ERR_VDEC_NOT_SUPPORT	该操作或者功能不支持。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 通道号不能超出最大的通道号范围。
- 创建解码通道前，需要保证解码通道未创建或者已销毁，否则会返回错误码MI_ERR_VDEC_CHN_EXIST。
- 如果设置eDpbBufMode为E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF模式或者E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF模式，解码器只保证支持正常解码Sstar编码器编出的码流，对第三方编码器编出的码流不保证完全支持；另外，eDpbBufMode要与待解码的码流参考帧个数一致。
- 如果需要解码B帧码流，则需要设置bDisableLowLatency为TRUE，以保证解码器按显示顺序输出解码后的图像；否则，输出的图像会有抖动，不连贯的问题。

➤ 举例

```
MI_S32 StartVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
    MI_VDEC_CHN VdecChn = 0;
    MI_VDEC_DisplayMode_e eDisplayMode = E_MI_VDEC_DISPLAY_MODE_MAX;
    MI_VDEC_ChnAttr_t stChnAttr;
    MI_VDEC_OutputPortAttr_t stOutputPortAttr;

    memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));
    memset(&stOutputPortAttr, 0x0, sizeof(MI_VDEC_OutputPortAttr_t));

    VdecChn = 0;
    stChnAttr.eCodecType = E_MI_VDEC_CODEC_TYPE_H264;
    stChnAttr.u32PicWidth = 1920;
    stChnAttr.u32PicHeight = 1080;
    stChnAttr.eVideoMode = E_MI_VDEC_VIDEO_MODE_FRAME;
    stChnAttr.u32BufSize = 1024*1024;
    stChnAttr.eDpbBufMode = E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF;
    stChnAttr.stVdecVideoAttr.u32RefFrameNum = 1;
    stChnAttr.u32Priority = 0;
    stChnAttr.stVdecVideoAttr.stErrHandlePolicy.bUseCusPolicy = FALSE;
    stChnAttr.stVdecVideoAttr.bDisableLowLatency = FALSE;
```



```
s32Ret = MI_VDEC_CreateChn(VdecDev, VdecChn, &stChnAttr);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_CreateChn failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

eDisplayMode = E_MI_VDEC_DISPLAY_MODE_PLAYBACK;
s32Ret = MI_VDEC_SetDisplayMode (VdecDev, VdecChn, eDisplayMode);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_SetDisplayMode failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

s32Ret = MI_VDEC_StartChn(VdecDev, VdecChn);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_StartChn failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

stOutputPortAttr.u16Width = 640;
stOutputPortAttr.u16Height = 480;
s32Ret = MI_VDEC_SetOutputPortAttr(VdecDev, VdecChn, &stOutputPortAttr);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_SetOutputPortAttr failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

stCropCfg.bEnable = TRUE;
stCropCfg.stRect.u16X = 0;
stCropCfg.stRect.u16Y = 0;
stCropCfg.stRect.u16Width = 720;
stCropCfg.stRect.u16Height = 576;
s32Ret = MI_S32 MI_VDEC_SetDestCrop(VdecDev, VdecChn, &stCropCfg);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_SetDestCrop failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

return MI_SUCCESS;
```

```
}

```

- 相关主题
- 无

2.4. MI_VDEC_DestroyChn

- 功能

销毁视频解码通道。

- 定义

```
MI_S32 MI_VDEC_DestroyChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);
```

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_FAILED	失败。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_NOT_STOP	销毁通道前未停止解码。
MI_ERR_VDEC_CHN_UNEXIST	通道不存在。
MI_ERR_VDEC_NOT_SUPPORT	该操作或者功能不支持
MI_ERR_VDEC_NULL_PTR	参数为空指针。

- 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

- ※ 注意

- 销毁解码通道前必须保证通道已创建，否则会返回通道未创建的错误码MI_ERR_VDEC_CHN_UNEXIST。
- 销毁解码通道前需要先停止解码，否则会返回通道未停止解码的错误码MI_ERR_VDEC_CHN_NOT_STOP

- 举例

```
MI_S32 StopVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
}
```

```

MI_VDEC_CHN VdecChn = 0;

// Destroy send stream thread
...

s32Ret = MI_VDEC_StopChn(VdecDev, VdecChn);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_StopChn failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

s32Ret = MI_VDEC_DestroyChn(VdecDev, VdecChn);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_DestroyChn failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

return MI_SUCCESS;
}

```

- 相关主题
无。

2.5. MI_VDEC_StartChn

- 功能
开始通道解码。
- 定义
MI_S32 MI_VDEC_StartChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围: [0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围: [0, MI_VDEC_MAX_CHN_NUM)。	输入

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_FAILED	失败。

返回值	描述
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_VDEC_CHN_NOT_STOP	通道未停止解码
MI_ERR_VDEC_NULL_PTR	参数为空指针。

➤ 依赖

- 头文件: mi_vdec.h
- 库文件: libmi_vdec.a/libmi_vdec.so

※ 注意

- 开始通道解码前必须保证通道已创建, 否则会返回通道未创建的错误码 MI_ERR_VDEC_CHN_UNEXIST。
- 开始通道解码之后, 才能调用 MI_VDEC_SendStream 发送码流成功。
- 重复调用该接口时, 会返回通道未停止解码的错误码 MI_ERR_VDEC_CHN_NOT_STOP。

➤ 举例

请参见[MI_VDEC_CreateChn](#)的举例。

➤ 相关主题

无。

2.6. MI_VDEC_StopChn

➤ 功能

停止通道解码。

➤ 语法

MI_S32 MI_VDEC_StopChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围: [0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围: [0, MI_VDEC_MAX_CHN_NUM)。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_FAILED	失败。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。

返回值	描述
MI_ERR_VDEC_NULL_PTR	参数为空指针。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_VDEC_CHN_NOT_START	通道未使能
MI_ERR_VDEC_BUSY	系统忙。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 调用该接口时，需要保证通道已创建，否则会报通道未创建的错误码MI_ERR_VDEC_CHN_UNEXIST。
- 调用该接口时，需要保证通道已使能，否则会报通道未使能的错误码MI_ERR_VDEC_CHN_NOT_START。
- 调用此接口后，调用发送码流的接口 [MI_VDEC_SendStream](#) 会返回失败。
- 重复调用该接口时，会返回通道未开始解码的错误码MI_ERR_VDEC_CHN_NOT_START。
- 解码B帧时，如果MI_VDEC后续绑定了MI_DISP模块，则在MI_VDEC停止通道解码前，需要保证MI_DISP模块先退出；否则，MI_VDEC退出时，会直接释放输出buffer，而MI_DISP此时还在访问MI_VDEC的输出buffer，会引起内存等问题；如果MI_VDEC在退出时，发现有buffer未还回，会报错MI_ERR_VDEC_BUSY。

➤ 举例

请参见[MI_VDEC_DestroyChn](#)的举例。

➤ 相关主题

无。

2.7. MI_VDEC_GetChnAttr

➤ 功能

获取视频解码通道属性。

➤ 定义

```
MI_S32 MI_VDEC_GetChnAttr(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
MI\_VDEC\_ChnAttr\_t *pstChnAttr);
```

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
pstChnAttr	解码通道属性指针。参数类型： MI_VDEC_ChnAttr_t 。	输出

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_NULL_PTR	输入参数为空指针。
MI_ERR_VDEC_CHN_UNEXIST	通道不存在。

➤ 依赖

- 头文件: mi_vdec.h
- 库文件: libmi_vdec.a/libmi_vdec.so

※ 注意

- 获取通道属性前必须保证通道已创建, 否则会返回通道未创建的错误码MI_ERR_VDEC_CHN_UNEXIST。

➤ 举例

无。

➤ 相关主题

无。

2.8. MI_VDEC_GetChnStat

➤ 功能

查询解码通道状态。

➤ 语法

MI_S32 MI_VDEC_GetChnStat (MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
[MI_VDEC_ChnStat_t](#) *pstChnStat);

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围: [0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	解码通道号。 取值范围: [0, MI_VDEC_MAX_CHN_NUM)。	输入
pstChnStat	解码通道状态结构体指针。	输出

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或已销毁。
MI_ERR_VDEC_NULL_PTR	输入参数为空指针。

- 依赖
 - 头文件: mi_vdec.h
 - 库文件: libmi_vdec.a/libmi_vdec.so
- ※ 注意
 - 查询解码通道状态前必须保证通道已创建, 否则会返回通道未创建的错误码MI_ERR_VDEC_CHN_UNEXIST。
- 举例

请参见[MI_VDEC_SendStream](#)的举例。
- 相关主题

无。

2.9. MI_VDEC_FlushChn

- 功能

清理解码通道缓存数据。
- 定义


```
MI_S32 MI_VDEC_FlushChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);
```
- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围: [0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围: [0, MI_VDEC_MAX_CHN_NUM)。	输入

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_FAILED	失败。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或已销毁。
MI_ERR_VDEC_CHN_NOT_START	通道未使能。
MI_ERR_VDEC_NULL_PTR	参数为空指针。
MI_ERR_VDEC_BUSY	解码器忙。

- 依赖
 - 头文件: mi_vdec.h
 - 库文件: libmi_vdec.a/libmi_vdec.so
- ※ 注意
 - 该接口可用于解码过程中, 切换到其他GOP时, 清理解码通道缓存的数据, 从而使解码继续进行。

- 举例
无。
- 相关主题
无。

2.10. MI_VDEC_ResetChn

- 功能
复位解码通道。
- 定义
MI_S32 MI_VDEC_ResetChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);
- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围: [0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围: [0, MI_VDEC_MAX_CHN_NUM)。	输入

- 返回值
- | 返回值 | 描述 |
|---------------------------|--|
| MI_SUCCESS | 成功。 |
| MI_ERR_VDEC_FAILED | 失败。 |
| MI_ERR_VDEC_INVALID_DEVID | 设备 ID 超出合法范围。 |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围。 |
| MI_ERR_VDEC_CHN_UNEXIST | 通道未创建或已销毁。 |
| MI_ERR_VDEC_NOT_PERM | 无法重置通道, 操作不能与 MI_VDEC_SendStream 同时调用。 |
| MI_ERR_VDEC_NOT_SUPPORT | 该操作或者功能不支持。 |

- 依赖
 - 头文件: mi_vdec.h
 - 库文件: libmi_vdec.a/libmi_vdec.so

- ※ 注意
 - 该接口暂不支持。

- 举例
无。
- 相关主题
无。

2.11. MI_VDEC_SetChnParam

➤ 功能

设置解码通道参数。

➤ 语法

```
MI_S32 MI_VDEC_SetChnParam(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,  
MI\_VDEC\_ChnParam\_t* pstChnParam);
```

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
pstChnParam	通道参数。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或已销毁。
MI_ERR_VDEC_ILLEGAL_PARAM	参数不合法。
MI_ERR_VDEC_NOT_PERM	非法操作，操作前需调用 MI_VDEC_StopChn 停止通道解码。
MI_ERR_VDEC_NOT_SUPPORT	该操作或者功能不支持。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 该接口暂不支持。

➤ 举例

无。

➤ 相关主题

无。

2.12. MI_VDEC_GetChnParam

➤ 功能

获取解码通道参数。

➤ 语法

```
MI_S32 MI_VDEC_GetChnParam(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,  
MI_VDEC_ChnParam_t* pstChnParam);
```

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
pstChnParam	通道参数。	输出

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或已销毁。
MI_ERR_VDEC_ILLEGAL_PARAM	参数不合法。
MI_ERR_VDEC_NOT_SUPPORT	该操作或者功能不支持。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 该接口暂不支持。

➤ 举例

无。

➤ 相关主题

无。

2.13. MI_VDEC_SendStream

➤ 功能

向解码通道发送码流数据。

➤ 语法

```
MI_S32 MI_VDEC_SendStream(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,  
MI\_VDEC\_VideoStream\_t *pstVideoStream, MI_S32 s32MilliSec);
```

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
pstVideoStream	解码码流数据指针。参考 MI_VDEC_VideoStream_t 定义。	输入
s32MilliSec	设定推送数据超时时间参数。取值范围： -1：阻塞。 0：非阻塞。 正值：超时时间，单位为毫秒。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_FAILED	失败。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_NOT_START	通道未使能。
MI_ERR_VDEC_ILLEGAL_PARAM	非法参数。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_VDEC_BUF_FULL	解码前的码流缓冲区满。
MI_ERR_VDEC_NOMEM	分配内存失败，如系统内存不足。
MI_ERR_VDEC_NOT_CONFIG	使用前未配置。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 暂不支持按流发送。
- 按帧传送时，每次需完整传送一帧数据。
- 若用户在送流时，设置pstVideoStream->u64PTS = -2，则在解码后，VDEC会丢弃该帧。用户可根据需要进行设置。

➤ 举例

每次传送完整一帧数据。以帧为单位，如果当前数据帧传送失败，需要重新传送。

```
MI_S32 VdecSendStream(void)
{
```

```
MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
MI_VDEC_DEV VdecDev = 0;
MI_VDEC_CHN VdecChn = 0;
MI_S32 s32MilliSec = 0;
MI_VDEC_VideoStream_t stVideoStream;
MI_VDEC_ChnAttr_t stChnAttr;
MI_VDEC_ChnStat_t stChnStat;

do{
    //Check if you need stop sending stream
    if(bStop)
    {
        break;
    }

    memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));
    s32Ret = MI_VDEC_GetChnAttr(VdecDev, VdecChn, &stChnAttr);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_GetChnAttr failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    memset(&stChnStat, 0x0, sizeof(MI_VDEC_ChnStat_t));
    s32Ret = MI_VDEC_GetChnStat(VdecDev, VdecChn, &stChnStat);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_GetChnStat failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    //suggest to check chn status
    if(stChnAttr.u32Bufsize - stChnStat.u32LeftStreamBytes < u32StreamSize)
    {
        continue;
    }

    memset(&stVideoStream, 0x0, sizeof(MI_VDEC_VideoStream_t));
    stVideoStream.pu8Addr = pu8StreamBuf;
    stVideoStream.u32Len = u32StreamSize;
    stVideoStream.u64PTS = u64StreamPts;
    stVideoStream.bEndOfFrame = TRUE;
    stVideoStream.bEndOfStream = FALSE;
    s32MilliSec = 0;           //0ms
```

```
s32Ret = MI_VDEC_SendStream(VdecDev, VdecChn, &stVideoStream, s32MilliSec);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_SendStream failed, s32Ret: 0x%x.\n", s32Ret);
    continue;
}
}while(!ShouldStop);

return MI_SUCCESS;
}
```

- 相关主题
无。

2.14. MI_VDEC_PauseChn

- 功能
暂停通道解码。
- 语法
MI_S32 MI_VDEC_PauseChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_VDEC_CHN_NOT_START	通道未使能或者已禁用。

- 依赖
 - 头文件：mi_vdec.h
 - 库文件：libmi_vdec.a/libmi_vdec.so

- ※ 注意

- 调用该接口后，解码器停止解码；此时，上层可以继续调用MI_VDEC_SendStream接口送流，但是码流buffer满时，MI_VDEC_SendStream会返回错误码MI_ERR_VDEC_BUF_FULL。

- 该接口可以重复调用，不会报错。

➤ 举例

```
MI_S32 PauseChn(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
    MI_VDEC_CHN VdecChn = 0;
    MI_U32 u32RefreshCnt = 0;

    //While decoding...

    s32Ret = MI_VDEC_PauseChn(VdecDev, VdecChn);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_PauseChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    //refresh 10 times
    while (u32RefreshCnt < 10)
    {
        s32Ret = MI_VDEC_RefreshChn(VdecDev, VdecChn);
        if(MI_SUCCESS != s32Ret)
        {
            printf("MI_VDEC_RefreshChn failed, s32Ret: 0x%x.\n", s32Ret);
            break;
        }
        u32RefreshCnt ++;
        usleep(100*1000);
    }

    s32Ret = MI_VDEC_ResumeChn(VdecDev, VdecChn);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_ResumeChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }
    return MI_SUCCESS;
}
```

➤ 相关主题

无。

2.15. MI_VDEC_RefreshChn

➤ 功能

刷新通道，再次解码当前帧。

➤ 语法

```
MI_S32 MI_VDEC_RefreshChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);
```

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_VDEC_CHN_NOT_START	通道未使能或者已禁用。
MI_ERR_VDEC_NOT_DISABLE	通道未暂停。
MI_ERR_VDEC_BUF_EMPTY	码流buffer为空。
MI_ERR_VDEC_CHN_NO_CONTENT	码流buffer无有效数据。
MI_ERR_VDEC_BUSY	解码器为BUSY状态，暂时不能refresh channel。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

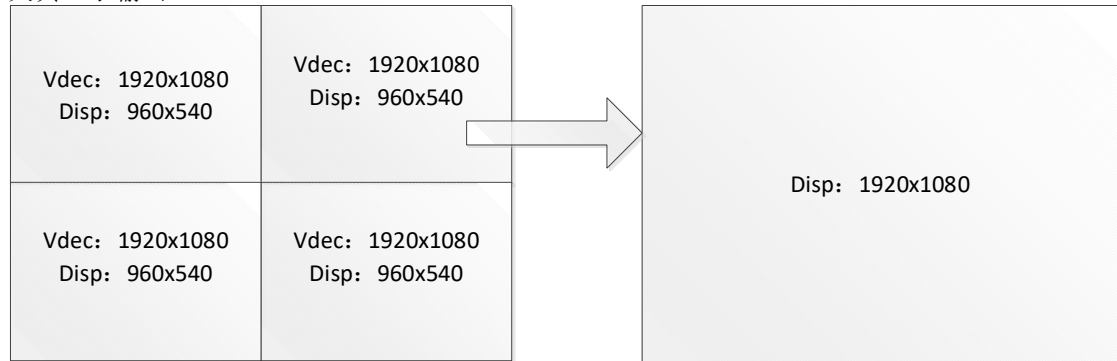
- 调用该接口时，需要保证已调用MI_VDEC_PauseChn停止解码，否则会报错误码MI_ERR_VDEC_NOT_DISABLE。
- 每调用一次该接口，会触发解码器再次解码当前帧；可以多次调用，同时可以配合MI_VDEC_SetOutputPortAttr调整缩放倍数，以适应不同的场景需求。
- 用户在调用该接口时，需要保证码流buffer中还有待解码的数据，否则不能完成refresh动作。
- 用户连续多次调用该接口时，如果上一次的refresh或step task未完成，则接口会返回错误码MI_ERR_VDEC_BUSY。
- 解码B帧时，不支持此功能。

➤ 举例

请参见[MI_VDEC_PauseChn](#)和[MI_VDEC_StepChn](#)的举例。

- 实现暂停时电子放大与抓图功能，目前只有Tiramisu和Muffin芯片支持此功能。

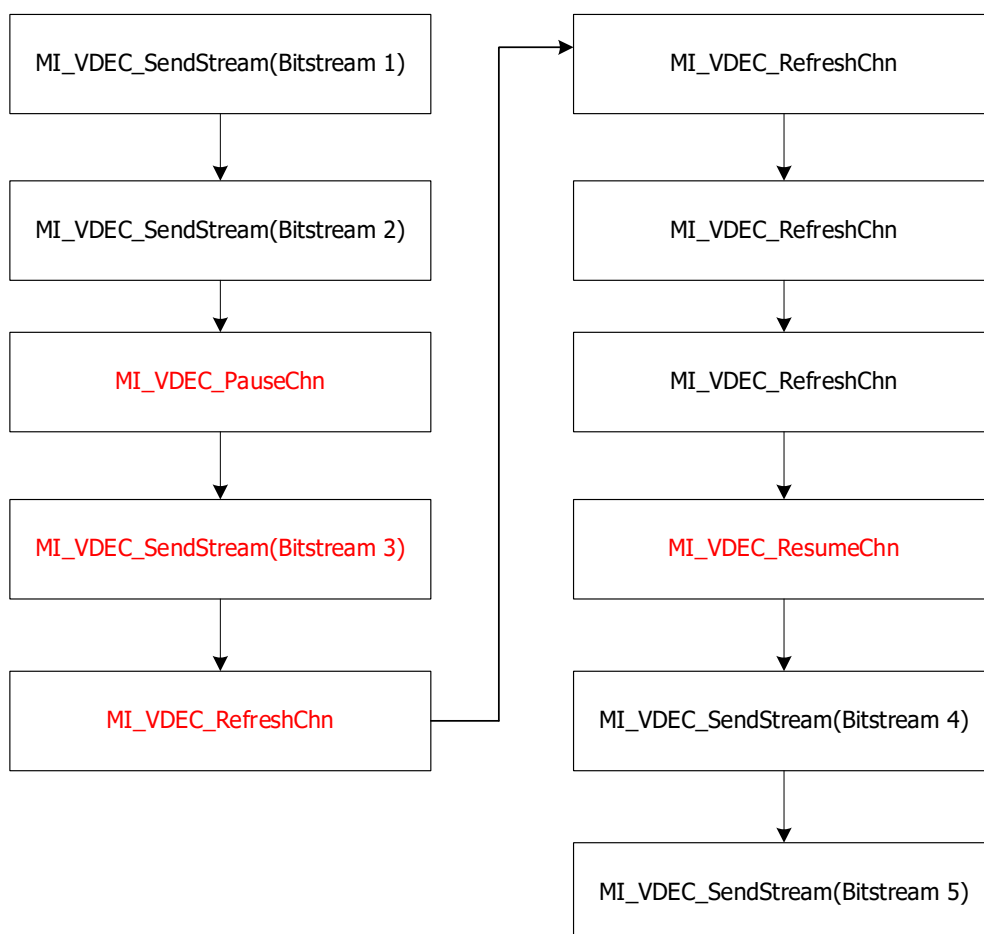
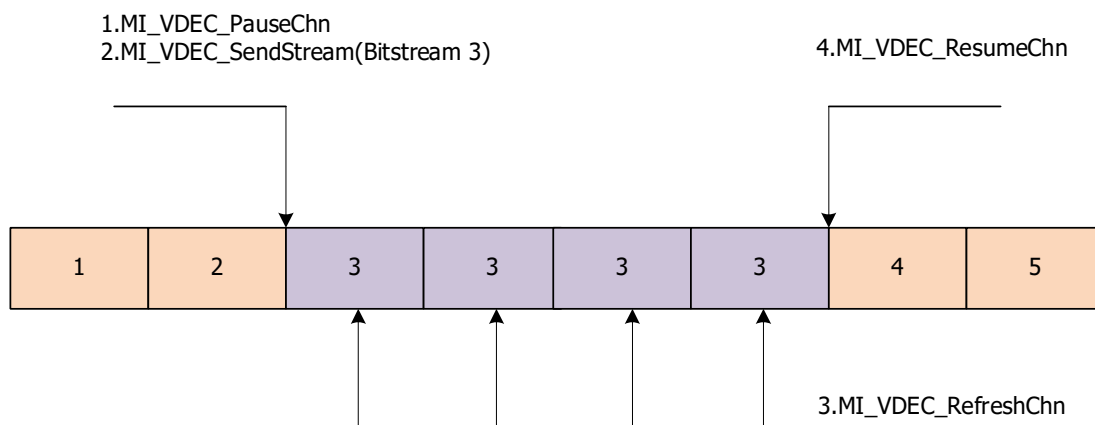
常见应用场景举例：假设MI_DISP在预览模式下输出分辨率为1920x1080，4通道都为1080P的源码流同时解码预览时，每个通道输出分辨率设置为960x540输出显示。如果暂停解码，然后将通道2切换至全屏预览，MI_DISP会将960x540的图像拉伸至1920x1080全屏显示，此时图像数据会失真。此时可以通过调用[MI_VDEC_RefreshChn](#)获取到当前暂停的数据帧的1920x1080的图像进行无失真显示输出。



通道要实现暂停电子放大功能，依赖于[MI_VDEC_PauseChn](#)，[MI_VDEC_RefreshChn](#)，[MI_VDEC_ResumeChn](#)以及[MI_VDEC_SendStream](#)组合，目的是为相同的码流数据帧能获得不同分辨率的图像的功能，如下，图像3需要重复输出4次。

API组合调用时序：

- Step 1: 调用[MI_VDEC_SendStream](#)推送数据帧 ‘1’和 ‘2’的码流数据。
- Step 2: 调用[MI_VDEC_PauseChn](#)暂停解码器，进入解码输出重复图像帧模式。
- Step 3: 调用[MI_VDEC_SendStream](#)推送需要重复输出图像 ‘3’的码流数据到解码器。
- Step 4: 重复调用[MI_VDEC_RefreshChn](#)，重复输出图像 ‘3’，每调用1次将输出1帧图像，调用次数无限制。当前示例调用4次。
- Step 5: 调用[MI_VDEC_ResumeChn](#)退出重复解码相同图像帧模式。
- Step 6: 调用[MI_VDEC_SendStream](#)推送数据帧 ‘4’和 ‘5’的码流数据。



➤ 相关主题
无。

2.16. MI_VDEC_ResumeChn

➤ 功能

恢复通道解码。

➤ 语法

```
MI_S32 MI_VDEC_ResumeChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);
```

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_VDEC_CHN_NOT_START	通道未使能或者已禁用。
MI_ERR_VDEC_BUSY	解码器为BUSY状态，暂时不能resume channel。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 调用该接口时，需要保证通道已创建，否则会报通道未创建的错误码MI_ERR_VDEC_CHN_UNEXIST。
- 调用该接口时，需要保证通道已使能，否则会报通道未使能的错误码MI_ERR_VDEC_CHN_NOT_START。
- 允许重复调用该接口。
- 调用该接口时，如果还有refresh或step task未完成，则接口会返回错误码MI_ERR_VDEC_BUSY。

➤ 举例

请参见[MI_VDEC_PauseChn](#)和[MI_VDEC_StepChn](#)的举例。

➤ 相关主题

无。

2.17. MI_VDEC_StepChn

➤ 功能

通道单帧解码。

➤ 语法

MI_S32 MI_VDEC_StepChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_VDEC_CHN_NOT_START	通道未使能或者已禁用。
MI_ERR_VDEC_BUF_EMPTY	码流buffer为空。
MI_ERR_VDEC_CHN_NO_CONTENT	码流buffer无有效数据。
MI_ERR_VDEC_BUSY	解码器为busy状态，暂时不能step channel。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 每调用一次该接口，会触发解码器解码下一帧；可以多次调用，同时可以配合MI_VDEC_SetOutputPortAttr调整缩放倍数，以适应不同的场景需求。
- 用户在调用该接口时，需要保证码流buffer中还有待解码的数据，否则不能完成step动作。
- 用户连续多次调用该接口时，如果上一次的refresh或step task未完成，则接口会返回错误码MI_ERR_VDEC_BUSY。

➤ 举例

```
Case 1:
MI_S32 StepChn(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
    MI_VDEC_CHN VdecChn = 0;

    //While decoding...

    s32Ret = MI_VDEC_PauseChn(VdecDev, VdecChn);
    if (MI_SUCCESS != s32Ret)
```

```

{
    printf("MI_VDEC_PauseChn failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

/// 此处送流过程也可以用另起独立的线程调用 MI_VDEC_SendStream 替代
s32Ret = MI_VDEC_SendStream(VdecDev, VdecChn, &stVdecStream, 0);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_SendStream failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

s32Ret = MI_VDEC_StepChn(VdecDev, VdecChn);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_StepChn failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

usleep(20*1000);

s32Ret = MI_VDEC_ResumeChn(VdecDev, VdecChn);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_ResumeChn failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

return MI_SUCCESS;
}

```

Case 2:

MI_S32 refresh_and_step_chn(void)

```

{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
    MI_VDEC_CHN VdecChn = 0;
    MI_U32 u32LoopCnt = 0;

    //While decoding...

    s32Ret = MI_VDEC_PauseChn(VdecDev, VdecChn);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_PauseChn failed, s32Ret: 0x%x.\n", s32Ret);
    }
}

```

```
    return s32Ret;
}

/// 此处送流过程也可以用另起独立的线程调用 MI_VDEC_SendStream 替代
s32Ret = MI_VDEC_SendStream(VdecDev, VdecChn, &stVdecStream, 0);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_SendStream failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

/*Loop 10 times*/
While(u32LoopCnt < 10)
{
    s32Ret = MI_VDEC_RefreshChn(VdecDev, VdecChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_RefreshChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }
    usleep(20*1000);

    /// 此处送流过程也可以用另起独立的线程调用 MI_VDEC_SendStream 替代
    s32Ret = MI_VDEC_SendStream(VdecDev, VdecChn, &stVdecStream, 0);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_SendStream failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    s32Ret = MI_VDEC_StepChn(VdecDev, VdecChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_StepChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }
    usleep(20*1000);
    u32LoopCnt ++;
}

s32Ret = MI_VDEC_ResumeChn(VdecDev, VdecChn);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_ResumeChn failed, s32Ret: 0x%x.\n", s32Ret);
```

```

    return s32Ret;
}
return MI_SUCCESS;
}

```

- 相关主题
- 无。

2.18. MI_VDEC_GetUserData

- 功能

获取解码通道的用户数据。

- 语法

MI_S32 MI_VDEC_GetUserData(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
[MI_VDEC_UserData_t](#) *pstUserData, MI_S32 s32MilliSec);

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
s32MilliSec	获取用户数据超时定义。取值范围： -1：阻塞。 0：非阻塞。 正值：推送数据超时时间，以 毫秒为单位。	输入
pstUserData	获取的解码用户数据，参考 MI_VDEC_UserData_t 定义	输出

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_ILLEGAL_PARAM	非法参数。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_VDEC_NOT_PERM	非法操作，如通道未准备好。
MI_ERR_VDEC_BUF_FULL	解码后的数据帧缓冲区满。

- 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

- ※ 注意
 - 该接口暂不支持。
- 举例
 - 无。
- 相关主题
 - 无。

2.19. MI_VDEC_ReleaseUserData

- 功能
 - 释放用户数据。
- 语法


```
MI_S32 MI_VDEC_ReleaseUserData(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
MI_VDEC_UserData_t * pstUserData);
```

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
pstUserData	解码后的用户数据指针，由 MI_VDEC_GetUserData 接口获取。	输入

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_ILLEGAL_PARAM	非法参数。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_VDEC_NOT_PERM	非法操作，如通道未准备好。

- 依赖
 - 头文件：mi_vdec.h
 - 库文件：libmi_vdec.a/libmi_vdec.so

- ※ 注意
 - 该接口暂不支持。
- 举例
 - 无。

- 相关主题
无。

2.20. MI_VDEC_SetDisplayMode

- 功能
设置显示模式。

- 语法
MI_S32 MI_VDEC_SetDisplayMode(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
[MI_VDEC_DisplayMode_e](#) eDisplayMode);

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
eDisplayMode	显示模式枚举。参考 MI_VDEC_DisplayMode_e 定义。	输入

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或已销毁。
MI_ERR_VDEC_ILLEGAL_PARAM	非法参数。

- 依赖
 - 头文件：mi_vdec.h
 - 库文件：libmi_vdec.a/libmi_vdec.so

- ※ 注意

- 设置显示模式前必须保证通道已创建，否则会返回通道未创建的错误码MI_ERR_VDEC_CHN_UNEXIST。
- 帧率控制机制优化后，MI_VDEC无需丢帧，因此该接口目前设置无效。
- 预览模式（E_MI_VDEC_DISPLAY_MODE_PREVIEW）：为了保持解码图像预览的实时性，MI_VDEC会在帧堆积到一定程度时，立刻丢弃解码后的图像，以快速清理码流buffer中堆积的数据，保证对实时接收到的码流立刻进行解码。
- 回放模式（E_MI_VDEC_DISPLAY_MODE_PLAYBACK）：为了保持解码图像的连贯性，MI_VDEC在解码过程中，即使出现了帧堆积的现象，也不会丢弃解码后的图像，避免显示图像卡顿，不连贯。

- 举例
请参见[MI_VDEC_CreateChn](#)的举例。

- 相关主题
无。

2.21. MI_VDEC_GetDisplayMode

- 功能

获取显示模式。

- 语法

MI_S32 MI_VDEC_GetDisplayMode(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, [MI_VDEC_DisplayMode_e](#) *peDisplayMode);

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
peDisplayMode	显示模式枚举指针，参考 MI_VDEC_DisplayMode_e 定义。	输入

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_NULL_PTR	输入参数为空指针。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或已销毁。

- 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

- ※ 注意

- 设置显示模式前必须保证通道已创建，否则会返回通道未创建的错误码 MI_ERR_VDEC_CHN_UNEXIST。

- 举例

无。

- 相关主题
无。

2.22. MI_VDEC_SetOutputPortAttr

➤ 功能

设置解码通道输出端口属性。

➤ 语法

```
MI_S32 MI_VDEC_SetOutputPortAttr (MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
MI_VDEC_OutputPortAttr_t *pstOutputPortAttr);
```

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
pstOutputPortAttr	输出端口属性，数据类型： MI_VDEC_OutputPortAttr_t	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或已销毁。
MI_ERR_VDEC_ILLEGAL_PARAM	输入非法参数。
MI_ERR_VDEC_NULL_PTR	输入参数空指针。
MI_ERR_VDEC_NOT_SUPPORT	该操作或者功能不支持。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 若通道未创建，则返回错误码MI_ERR_VDEC_CHN_UNEXIST。
- 缩放宽和高需要按2对齐。
- 缩放范围[1/8, 1]。
- 不支持放大。
- 若裁剪和缩放同时开启，则缩放宽和高不能大于裁剪的宽和高。

➤ 举例

请参见[MI_VDEC_CreateChn](#)的举例。

➤ 相关主题

无。

2.23. MI_VDEC_GetOutputPortAttr

➤ 功能

获取解码通道输出端口属性。

➤ 语法

```
MI_S32 MI_VDEC_GetOutputPortAttr(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
MI_VDEC_OutputPortAttr_t *pstOutputPortAttr);
```

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	视频解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
pstOutputPortAttr	输出端口属性，数据类型： MI_VDEC_OutputPortAttr_t	输出

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或已销毁。
MI_ERR_VDEC_NULL_PTR	输入参数空指针
MI_ERR_VDEC_NOT_SUPPORT	该操作或者功能不支持。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 若通道未创建，则返回错误码MI_ERR_VDEC_CHN_UNEXIST

➤ 举例

无。

➤ 相关主题

无。

2.24. MI_VDEC_SetOutputPortLayoutMode

➤ 功能

设置输出端口Buffer模式。

➤ 语法

```
MI_S32 MI_VDEC_SetOutputPortLayoutMode(MI_VDEC_DEV VdecDev,  
MI\_VDEC\_OutbufLayoutMode\_e eBufTileMode);
```

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
eBufTileMode	输出Buffer模式，数据类型： MI_VDEC_OutbufLayoutMode_e	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_FAILED	失败。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_NOT_INIT	设备未初始化。
MI_ERR_VDEC_ILLEGAL_PARAM	非法参数。
MI_ERR_VDEC_BUSY	系统忙。

➤ 依赖

- 头文件：mi_vdec.h
- 库文件：libmi_vdec.a/libmi_vdec.so

※ 注意

- 若设备未初始化，则返回错误码MI_ERR_VDEC_NOT_INIT。
- 该接口需要在开始解码前调用，否则会报系统忙的的错误码MI_ERR_VDEC_BUSY。

➤ 举例

```
MI_S32 StartVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
    MI_VDEC_CHN VdecChn = 0;
    MI_VDEC_OutbufLayoutMode_e eBufTileMode = E_MI_VDEC_OUTBUF_LAYOUT_MAX;
    MI_VDEC_ChnAttr_t stChnAttr;

    memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));

    eBufTileMode = E_MI_VDEC_OUTBUF_LAYOUT_AUTO;
    s32Ret = MI_VDEC_SetOutputPortLayoutMode(VdecDev, eBufTileMode);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_SetOutputPortLayoutMode failed, s32Ret: 0x%x.\n", s32Ret);
    }
}
```

```

    return s32Ret;
}

VdecChn = 0;
stChnAttr.eCodecType = E_MI_VDEC_CODEC_TYPE_H264;
stChnAttr.u32PicWidth = 1920;
stChnAttr.u32PicHeight = 1080;
stChnAttr.eVideoMode = E_MI_VDEC_VIDEO_MODE_FRAME;
stChnAttr.u32BufSize = 1024*1024;
stChnAttr.eDpbBufMode = E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF;
stChnAttr.stVdecVideoAttr.u32RefFrameNum = 1;
stChnAttr.u32Priority = 0;

s32Ret = MI_VDEC_CreateChn(VdecDev, VdecChn, &stChnAttr);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_CreateChn failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

return MI_SUCCESS;
}

```

- 相关主题
无。

2.25. MI_VDEC_GetOutputPortLayoutMode

- 功能

获取输出端口Buffer模式。

- 语法

```
MI_S32 MI_VDEC_GetOutputPortLayoutMode(MI_VDEC_DEV VdecDev,
MI_VDEC_OutbufLayoutMode_e *peBufTileMode);
```

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
eBufTileMode	输出Buffer模式，数据类型： MI_VDEC_OutbufLayoutMode_e	输出

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_NOT_INIT	设备未初始化。
MI_ERR_VDEC_NULL_PTR	输入参数空指针

➤ 依赖

- 头文件: mi_vdec.h
- 库文件: libmi_vdec.a/libmi_vdec.so

※ 注意

- 若设备未初始化, 则返回错误码MI_ERR_VDEC_NOT_INIT

➤ 举例

无。

➤ 相关主题

无。

2.26. MI_VDEC_SetDestCrop

➤ 功能

设置解码通道裁剪属性。

➤ 语法

```
MI_S32 MI_VDEC_SetDestCrop(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
MI_VDEC_CropCfg_t *pstCropCfg);
```

➤ 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围: [0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	解码通道号。 取值范围: [0, MI_VDEC_MAX_CHN_NUM)。	输入
pstCropCfg	解码通道裁剪属性指针, 数据类型: MI_VDEC_CropCfg_t 。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或已销毁。
MI_ERR_VDEC_ILLEGAL_PARAM	非法参数。
MI_ERR_VDEC_NULL_PTR	输入参数空指针。

返回值	描述
MI_ERR_VDEC_NOT_SUPPORT	该操作或者功能不支持。

- 依赖
 - 头文件：mi_vdec.h
 - 库文件：libmi_vdec.a/libmi_vdec.so
- ※ 注意
 - 裁剪设置的坐标X和宽需要按16对齐，坐标Y和高需要按2对齐；
 - 若通道未创建，则返回错误码MI_ERR_VDEC_CHN_UNEXIST；
 - 若裁剪和缩放同时开启，则裁剪宽和高不能小于缩放的宽和高。

- 举例

请参见[MI_VDEC_CreateChn](#)的举例。

- 相关主题

无。

2.27. MI_VDEC_GetDestCrop

- 功能

获取解码通道裁剪属性。

- 语法


```
MI_S32 MI_VDEC_GetDestCrop(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
MI\_VDEC\_CropCfg\_t *pstCropCfg);
```

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围：[0, MI_VDEC_MAX_DEV_NUM)。	输入
VdecChn	解码通道号。 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。	输入
pstCropCfg	解码通道裁剪属性指针，数据类型： MI_VDEC_CropCfg_t 。	输出

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围。
MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_VDEC_CHN_UNEXIST	通道未创建或已销毁。
MI_ERR_VDEC_NULL_PTR	输入参数空指针。
MI_ERR_VDEC_NOT_SUPPORT	该操作或者功能不支持。

- 依赖
 - 头文件: `mi_vdec.h`
 - 库文件: `libmi_vdec.a/libmi_vdec.so`
- ※ 注意
 - 若通道未创建, 则返回错误码`MI_ERR_VDEC_CHN_UNEXIST`;
- 举例

无。
- 相关主题

无。

2.28. MI_VDEC_SetChnErrHandlePolicy

- 功能

设置解码通道错误宏块的数据帧的输出策略。
- 语法


```
MI_S32 MI_VDEC_SetChnErrHandlePolicy(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
MI_VDEC_ErrHandlePolicy_t *pstErrHandlePolicy);
```

- 形参

参数名称	描述	输入/输出
VdecDev	解码设备号。 取值范围: <code>[0, MI_VDEC_MAX_DEV_NUM)</code> 。	输入
VdecChn	解码通道号。 取值范围: <code>[0, MI_VDEC_MAX_CHN_NUM)</code> 。	输入
pstErrHandlePolicy	设置错误宏块的数据帧的输出策略, 参考 MI_VDEC_ErrHandlePolicy_t 定义。	输入

- 返回值

返回值	描述
<code>MI_SUCCESS</code>	成功。
<code>MI_ERR_VDEC_INVALID_DEVID</code>	设备 ID 超出合法范围。
<code>MI_ERR_VDEC_INVALID_CHNID</code>	通道 ID 超出合法范围。
<code>MI_ERR_VDEC_CHN_UNEXIST</code>	通道未创建或已销毁。
<code>MI_ERR_VDEC_NULL_PTR</code>	输入参数空指针。
<code>MI_ERR_VDEC_NOT_SUPPORT</code>	该操作或者功能不支持。

- 依赖
 - 头文件: `mi_vdec.h`
 - 库文件: `libmi_vdec.a/libmi_vdec.so`
- ※ 注意
 - 若通道未创建, 则返回错误码`MI_ERR_VDEC_CHN_UNEXIST`。

- 举例
无。
- 相关主题
无。

3. 数据类型

视频解码相关数据类型、数据结构定义如下：

数据结构	说明
MI_VDEC_CodecType_e	定义解码类型
MI_VDEC_DPB_BufMode_e	定义DPB buffer模式
MI_VDEC_JpegFormat_e	定义Jpeg 图片数据格式
MI_VDEC_VideoMode_e	定义码流发送方式
MI_VDEC_ErrCode_e	定义错误码类型
MI_VDEC_DecodeMode_e	定义码流解码方式
MI_VDEC_OutputOrder_e	定义解码输出顺序方式
MI_VDEC_VideoFormat_e	定义解码图像数据格式
MI_VDEC_DisplayMode_e	定义显示模式
MI_VDEC_OutbufLayoutMode_e	定义输出buffer模式
MI_VDEC_InitParam_t	定义解码设备初始化参数结构体
MI_VDEC_ChnAttr_t	定义视频解码通道属性
MI_VDEC_JpegAttr_t	定义JPEG视频解码属性
MI_VDEC_VideoAttr_t	定义H264/H265视频解码属性
MI_VDEC_ChnStat_t	定义通道状态结构体
MI_VDEC_ChnParam_t	定义解码通道参数结构体
MI_VDEC_VideoStream_t	定义视频解码的码流结构体
MI_VDEC_UserData_t	定义用户数据结构体
MI_VDEC_OutputPortAttr_t	定义输出端口属性
MI_VDEC_ErrHandlePolicy_t	定义错误宏块处理策略结构体
MI_VDEC_CropCfg_t	定义裁剪属性

3.1. MI_VDEC_CodecType_e

- 说明
定义解码类型。
- 定义
typedef enum

```
{
    E_MI_VDEC_CODEC_TYPE_H264 = 0x0,      /* H264 */
    E_MI_VDEC_CODEC_TYPE_H265,           /* H265 */
    E_MI_VDEC_CODEC_TYPE_JPEG,           /* JPEG */
    E_MI_VDEC_CODEC_TYPE_MAX
} MI_VDEC_CodecType_e;
```

➤ 成员

成员名称	描述
E_MI_VDEC_CODEC_TYPE_H264	H264解码。
E_MI_VDEC_CODEC_TYPE_H265	H265解码。
E_MI_VDEC_CODEC_TYPE_JPEG	JPEG解码。

※ 注意事项

- 暂不支持用户设置JPEG解码。

➤ 相关数据类型及接口

无。

3.2. MI_VDEC_DPB_BufMode_e

➤ 说明

定义DPB buffer模式。

➤ 定义

```
typedef enum
{
    E_MI_VDEC_DPB_MODE_NORMAL=0,
    E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF=1,
    E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF=2,
    E_MI_VDEC_DPB_MODE_MAX
} MI_VDEC_DPB_BufMode_e;
```

➤ 成员

成员名称	描述
E_MI_VDEC_DPB_MODE_NORMAL	普通模式。
E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF	Inplace one buffer模式。
E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF	Inplace two buffer模式。

※ 注意事项

无。

➤ 相关数据类型及接口

无。

3.3. MI_VDEC_JpegFormat_e

➤ 说明

定义Jpeg图片数据格式。

➤ 定义

```
typedef enum
{
    E_MI_VDEC_JPEG_FORMAT_YCBCR400 = 0x0, /*YUV400*/
    E_MI_VDEC_JPEG_FORMAT_YCBCR420, /*YUV420*/
    E_MI_VDEC_JPEG_FORMAT_YCBCR422, /*YUV 422*/
    E_MI_VDEC_JPEG_FORMAT_YCBCR444, /*YUV 444*/
    E_MI_VDEC_JPEG_FORMAT_MAX
} MI_VDEC_JpegFormat_e;
```

➤ 成员

略。

※ 注意事项

- 目前暂不支持该参数相关功能。

➤ 相关数据类型及接口

无。

3.4. MI_VDEC_VideoMode_e

➤ 说明

定义码流发送方式。

➤ 定义

```
typedef enum
{
    E_MI_VDEC_VIDEO_MODE_STREAM = 0x0,
    E_MI_VDEC_VIDEO_MODE_FRAME,
    E_MI_VDEC_VIDEO_MODE_MAX
} MI_VDEC_VideoMode_e;
```

➤ 成员

成员名称	描述
E_MI_VIDEO_MODE_STREAM	按流方式发送码流。
E_MI_VIDEO_MODE_FRAME	按帧方式发送码流，以帧为单位。

※ 注意事项

- 暂不支持按流方式送码流。

➤ 相关数据类型及接口

无。

3.5. MI_VDEC_ErrCode_e

➤ 说明

定义错误码类型。

➤ 定义

```
typedef enum
{
    E_MI_VDEC_ERR_CODE_NONE = 0x0,
    E_MI_VDEC_ERR_CODE_UNKNOWN,
    E_MI_VDEC_ERR_CODE_MB_ERROR,
    E_MI_VDEC_ERR_CODE_REF_FRAME_ERROR,
    E_MI_VDEC_ERR_CODE_REF_FRAME_BUFF_NOT_ENOUGH,
    E_MI_VDEC_ERR_CODE_VCL_NOT_FOUND,
    E_MI_VDEC_ERR_CODE_OVER_PROFILE,
    E_MI_VDEC_ERR_CODE_OVER_LEVEL,
    E_MI_VDEC_ERR_CODE_OVER_MULTISLICE_NUM,
    E_MI_VDEC_ERR_CODE_ILLEGAL_ACCESS,
    E_MI_VDEC_ERR_CODE_FRMRATE_UNSUPPORT,
    E_MI_VDEC_ERR_CODE_DEC_TIMEOUT,
    E_MI_VDEC_ERR_CODE_OUT_OF_MEMORY,
    E_MI_VDEC_ERR_CODE_CODEC_TYPE_UNSUPPORT,
    E_MI_VDEC_ERR_CODE_ERR_SPS_UNSUPPORT,
    E_MI_VDEC_ERR_CODE_ERR_PPS_UNSUPPORT,
    E_MI_VDEC_ERR_CODE_REF_LIST_ERR,
    E_MI_VDEC_ERR_CODE_MAX
} MI_VDEC_ErrCode_e;
```

➤ 成员

成员名称	描述
E_MI_VDEC_ERR_CODE_NONE	无
E_MI_VDEC_ERR_CODE_UNKNOWN	未知错误。
E_MI_VDEC_ERR_CODE_MB_ERROR	宏块错误
E_MI_VDEC_ERR_CODE_REF_FRAME_ERROR	参考帧错误
E_MI_VDEC_ERR_CODE_REF_FRAME_BUFF_NOT_ENOUGH	参考帧分配个数不够
E_MI_VDEC_ERR_CODE_VCL_NOT_FOUND	无有效的图像数据
E_MI_VDEC_ERR_CODE_OVER_PROFILE	Profile不支持
E_MI_VDEC_ERR_CODE_OVER_LEVEL	Level不支持
E_MI_VDEC_ERR_CODE_OVER_MULTISLICE_NUM	视频帧Slice个数超出限制
E_MI_VDEC_ERR_CODE_ILLEGAL_ACCESS	不可访问，例如:未初始化或硬件出问题。

成员名称	描述
E_MI_VDEC_ERR_CODE_FRMRATE_UNSupport	帧率不支持
E_MI_VDEC_ERR_CODE_DEC_TIMEOUT	解帧超时
E_MI_VDEC_ERR_CODE_OUT_OF_MEMORY	内存不足
E_MI_VDEC_ERR_CODE_CODEEC_TYPE_UNSupport	解码类型不支持
E_MI_VDEC_ERR_CODE_ERR_SPS_UNSupport	不支持的SPS或SPS出错
E_MI_VDEC_ERR_CODE_ERR_PPS_UNSupport	不支持的PPS或PPS出错
E_MI_VDEC_ERR_CODE_REF_LIST_ERR	参考帧列表出错

※ 注意事项

- 目前暂不支持该参数相关功能。

➤ 相关数据类型及接口

无。

3.6. MI_VDEC_DecodeMode_e

➤ 说明

定义码流解码方式。

➤ 定义

```
typedef enum
{
    E_MI_VDEC_DECODE_MODE_ALL = 0x0,
    E_MI_VDEC_DECODE_MODE_I,
    E_MI_VDEC_DECODE_MODE_IP,
    E_MI_VDEC_DECODE_MODE_MAX
} MI_VDEC_DecodeMode_e;
```

➤ 成员

成员名称	描述
E_MI_VDEC_DECODE_MODE_ALL	解码IPB数据帧模式
E_MI_VDEC_DECODE_MODE_I	只解I帧模式
E_MI_VDEC_DECODE_MODE_IP	只解IP帧，跳过B帧

※ 注意事项

- 该参数暂不支持设置。

➤ 相关数据类型及接口

无。

3.7. MI_VDEC_OutputOrder_e

➤ 说明

定义解码输出顺序方式。

➤ 定义

```
typedef enum
{
    E_MI_VDEC_OUTPUT_ORDER_DISPLAY = 0x0,
    E_MI_VDEC_OUTPUT_ORDER_DECODE,
    E_MI_VDEC_OUTPUT_ORDER_MAX,
} MI_VDEC_OutputOrder_e;
```

➤ 成员

成员名称	描述
E_MI_VDEC_OUTPUT_ORDER_DISPLAY	按显示顺序输出数据帧
E_MI_VDEC_OUTPUT_ORDER_DECODE	按解帧顺序列输出数据帧

➤ 数据类型

- 目前内部默认按显示顺序输出数据帧，不支持用户设置。

➤ 相关数据类型及接口

无。

3.8. MI_VDEC_VideoFormat_e

➤ 说明

定义解码图像数据格式。

➤ 定义

```
typedef enum
{
    E_MI_VDEC_VIDEO_FORMAT_TILE = 0x0,
    E_MI_VDEC_VIDEO_FORMAT_REDUCE,
    E_MI_VDEC_VIDEO_FORMAT_MAX
} MI_VDEC_VideoFormat_e;
```

➤ 成员

成员名称	描述
E_MI_VDEC_VIDEO_FORMAT_TILE	TILE数据格式。
E_MI_VDEC_VIDEO_FORMAT_REDUCE	数据帧压缩模式，减少数据帧内存使用量。

※ 注意事项

- 当前数据类型不支持上层设定，只能返回支持数据类型。

➤ 相关数据类型及接口

无。

3.9. MI_VDEC_DisplayMode_e

➤ 说明

定义显示模式。

➤ 定义

```
typedef enum
{
    E_MI_VDEC_DISPLAY_MODE_PREVIEW = 0x0,
    E_MI_VDEC_DISPLAY_MODE_PLAYBACK,
    E_MI_VDEC_DISPLAY_MODE_MAX,
} MI_VDEC_DisplayMode_e;
```

➤ 成员

成员名称	描述
E_MI_DISPLAY_MODE_PREVIEW	预览模式。不参考PTS输出。
E_MI_DISPLAY_MODE_PLAYBACK	回放模式。参考PTS数值输出。

※ 注意事项

无。

➤ 相关数据类型及接口

无。

3.10. MI_VDEC_OutbufLayoutMode_e

➤ 说明

定义输出buffer模式。

➤ 定义

```
typedef enum
{
    E_MI_VDEC_OUTBUF_LAYOUT_AUTO = 0x0,
    E_MI_VDEC_OUTBUF_LAYOUT_LINEAR,
    E_MI_VDEC_OUTBUF_LAYOUT_TILE,
    E_MI_VDEC_OUTBUF_LAYOUT_MAX
} MI_VDEC_OutbufLayoutMode_e;
```

➤ 成员

成员名称	描述
E_MI_VDEC_OUTBUF_LAYOUT_AUTO	输出buffer自适应模式。
E_MI_VDEC_OUTBUF_LAYOUT_LINEAR	输出buffer为LINEAR模式。
E_MI_VDEC_OUTBUF_LAYOUT_TILE	输出buffer为TILE模式

※ 注意事项

- MI_DISP模块的旋转功能和MI_VDEC模块的TILE模式存在联动关系。如果MI_DISP模块需要开启旋转功能，则其前端送入的buffer必须为TILE格式。MI_VDEC模块有两种方式输出TILE格式的图像。
 - 1) 输出buffer模式为E_MI_VDEC_OUTBUF_LAYOUT_AUTO（默认）时，MI_DISP模块和MI_VDEC模块需要绑定；此时，由MI_DISP模块开启旋转，同时通过u64SidebandMsg告知MI_VDEC模块，然后MI_VDEC模块自动打开TILE模式，即可完成旋转功能的开启。如果MI_DISP模块关闭旋转，则同样通过u64SidebandMsg告知MI_VDEC模块，然后MI_VDEC模块再自动关闭TILE模式。该过程无需用户处理，由模块内部自动完成。
 - 2) 输出buffer模式为E_MI_VDEC_OUTBUF_LAYOUT_TILE时，MI_VDEC模块仅输出TILE格式的图像。MI_DISP模块必须开启旋转功能，才能正常显示TILE格式的图像。这种模式下，MI_VDEC模块和MI_DISP模块既可以直接绑定，也可由用户获取MI_VDEC模块的输出buffer，再发送给MI_DISP模块。
- 如果MI_VDEC模块需要输出LINEAR图像，有两种方式。
 - 1) 输出buffer模式为E_MI_VDEC_OUTBUF_LAYOUT_AUTO（默认）时，如果MI_VDEC模块和MI_DISP模块绑定，则MI_DISP模块关闭旋转，即可输出LINEAR图像。其他情况下，均输出LINEAR图像。
 - 2) 输出buffer模式为E_MI_VDEC_OUTBUF_LAYOUT_LINEAR时，MI_VDEC模块仅输出LINEAR图像。
- 输出buffer为TILE模式时，MI_VDEC模块输出buffer对齐方式如下表。

芯片	TILE 输出buffer对齐要求（BYTE）
Taiyaki	128x32
Takoyaki	128x32
Tiramisu	128x64
Muffin	128x64

因此，通过[MI_VDEC_OutputPortAttr_t](#)设置MI_VDEC模块输出buffer宽高时，也需要按TILE输出buffer对齐要求进行设置，否则会输出异常图像。如：TILE输出buffer对齐要求为128x32，那么[MI_VDEC_OutputPortAttr_t](#)设置MI_VDEC模块输出buffer的宽高也需要按128x32对齐。

- 相关数据类型及接口
无。

3.11. MI_VDEC_InitParam_t

- 说明
定义解码设备初始化参数结构体。
- 定义


```
typedef struct MI_VDEC_InitParam_s
{
    MI_U16 u16MaxWidth;
    MI_U16 u16MaxHeight;
} MI_VDEC_InitParam_t;
```
- 成员

成员名称	描述
------	----

u16MaxWidth	解码器支持的最大码流宽。参数类型: MI_U16
u16MaxHeight	解码器支持的最大码流高。参数类型: MI_U16

※ 注意事项

- VDEC根据硬件性能，内部默认设置了一组最大分辨率。如果用户需要解码超出默认的最大分辨率（如：4Kx3K），则需要在此修改最大码流的宽和高，以便可以正常解码。如果超出默认分辨率，解码帧率相应下降。

- 相关数据类型及接口
无。

3.12. MI_VDEC_ChnAttr_t

- 说明

定义视频解码通道属性。

- 定义

```
typedef struct MI_VDEC_ChnAttr_s
{
    MI_VDEC_CodecType_e eCodecType;
    MI_U32    u32BufSize;
    MI_U32    u32Priority;
    MI_U32    u32PicWidth;
    MI_U32    u32PicHeight;
    MI_VDEC_VideoMode_e eVideoMode;
    MI_VDEC_DPB_BufMode_e eDpbBufMode;
    union
    {
        MI_VDEC_JpegAttr_t stVdecJpegAttr;
        MI_VDEC_VideoAttr_t stVdecVideoAttr;
    };
} MI_VDEC_ChnAttr_t;
```

- 成员

成员名称	描述
eCodecType	解码类型枚举值。参数类型: MI_VDEC_CodecType_e 。
u32BufSize	码流缓存的大小。
u32Priority	通道优先级，取值范围为 1 ~ 255，值越大优先级越高。 注意：当前暂不支持此功能。
u32PicWidth	通道支持的解码图像最大宽（以像素为单位）。
u32PicHeight	通道支持的解码图像最大高（以像素为单位）。
eVideoMode	码流传送方式。 注意：当前只支持按帧。
eDpbBufMode	DPB buffer 模式。 注意：

成员名称	描述
	eDpbBufMode=E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF, 只适用于一个参考帧的码流; eDpbBufMode=E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF, 适用于两个参考帧的码流; 如果超过两个参考帧, 则需要设置为 eDpbBufMode=E_MI_VDEC_DPB_MODE_NORMAL。
stVdecJpegAttr	JPEG 通道的相关属性, 目前暂不支持该参数相关功能。
stVdecVideoAttr	除 JPEG 以外其它所支持类型的通道的相关属性

※ 注意事项

- 当设置了eDpbBufMode为E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF或者E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF时, u32RefFrameNum的值将无效;
- E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF与E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF为省内存模式, E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF解码器只分配1个参考帧, 因此也只能解码1个参考帧的码流, 同理, E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF最多只分配2个参考帧, 只能解码最多2个参考帧的码流。对于2个参考帧以上的码流, 只支持设置为E_MI_VDEC_DPB_MODE_NORMAL。例如: 设置为E_MI_VDEC_DPB_MODE_NORMAL模式时, 解码1个参考帧的码流, 解码器需要申请2个参考帧才能正常解码, 会比E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF省内存模式多申请1个参考帧。
- 当eDpbBufMode为E_MI_VDEC_DPB_MODE_NORMAL时, 需要设置u32RefFrameNum的值, 用于限制最大参考帧个数, 避免分配过多的frame buffer; 若u32RefFrameNum的值小于当前解码所需的参考帧个数, 则可能不解码或者解码花屏。
- 如果无法预先确定码流参考帧个数, 那么建议设置为E_MI_VDEC_DPB_MODE_NORMAL模式, 最大参考帧申请个数u32RefFrameNum设置为16。解码器会自动解析码流所需参考帧个数, 按解码码流所需参考帧个数进行申请。例如: u32RefFrameNum设置为16, 当前码流解码需要3个参考帧, 那么解码器将申请4个参考帧; 反之, 如果u32RefFrameNum设置为2, 当前解码需要3个参考帧的码流, 那么解码器最多只能申请到2个参考帧, 无法正常解码。

- 相关数据类型及接口
无。

3.13. MI_VDEC_JpegAttr_t

- 说明
定义JPEG视频解码属性。

- 定义

```
typedef struct MI_VDEC_JpegAttr_s
{
    MI_VDEC_JpegFormat_e eJpegFormat;
}MI_VDEC_JpegAttr_t;
```

- 成员

成员名称	描述
------	----

eJpegFormat	JPEG图片的存储格式。 取值范围是[0,E_MI_JPEG_FORMAT_MAX)。
-------------	--

※ 注意事项

- 目前暂不支持该参数相关功能。

- 相关数据类型及接口
无。

3.14. MI_VDEC_VideoAttr_t

➤ 说明

定义H264/H265视频解码属性。

➤ 定义

```
typedef struct MI_VDEC_VideoAttr_s
{
    MI_U32 u32RefFrameNum;
    MI_VDEC_ErrHandlePolicy_t stErrHandlePolicy;
    MI_BOOL bDisableLowLatency;
}MI_VDEC_VideoAttr_t;
```

➤ 成员

成员名称	描述
u32RefFrameNum	参考帧的数目。 取值范围：[0, 16]，以帧为单位。 参考帧的数目决定解码时需要的参考帧个数，影响内存使用量，根据实际情况设置合适的值。测试码流：推荐设为 2。
stErrHandlePolicy	设置错误宏块的数据帧的输出策略，参考 MI_VDEC_ErrHandlePolicy_t 定义。
bDisableLowLatency	是否开启B帧解码功能，默认：FALSE。

※ 注意事项

- 当设置了eDpbBufMode为E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF或者E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF时，u32RefFrameNum的值将无效；
- 当eDpbBufMode为E_MI_VDEC_DPB_MODE_NORMAL时，需要设置u32RefFrameNum的值，用于限制最大参考帧个数，避免分配过多的frame buffer；若u32RefFrameNum的值小于当前解码所需的参考帧个数，则可能不解码或者解码花屏。
- 若bDisableLowLatency设为TRUE，表示关闭低延时；这种情况适用于解码B帧，用于对B帧的输出帧做重排序，输出画面会有延时。另外，做重排序操作，也会增加内存使用量。
- 如果用户的场景无需解码B帧格式码流，则需要设置 bDisableLowLatency为FALSE。

- 相关数据类型及接口
无。

3.15. MI_VDEC_ChnStat_t

➤ 说明

定义通道状态结构体。

➤ 定义

```
typedef MI_VDEC_ChnStat_s
{
    MI_VDEC_CodecType_e eCodecType;
    MI_U32 u32LeftStreamBytes;
    MI_U32 u32LeftStreamFrames;
    MI_U32 u32LeftPics;
    MI_BOOL bChnStart;
    MI_U32 u32RecvStreamFrames;
    MI_U32 u32DecodeStreamFrames;
    MI_VDEC_ErrCode_e eErrCode;
} MI_VDEC_ChnStat_t;
```

➤ 成员

成员名称	描述
eCodecType	解码类型
u32LeftStreamBytes	码流 buffer 中待解码的 byte 数。
u32LeftStreamFrames	码流 buffer 中待解码的帧数。 -1 表示无效。 仅按帧发送时有效。
u32LeftPics	图像 buffer 中剩余的 pic 数目。
bChnStart	解码器是否已经启动接收码流。
u32RecvStreamFrames	码流 buffer 中已接收码流帧数。 -1 表示无效。 仅按帧发送时有效。
u32DecodeStreamFrames	码流 buffer 中已解码帧数。
eErrCode	解码错误码信息。

※ 注意事项
无。

➤ 相关数据类型及接口
无。

3.16. MI_VDEC_ChnParam_t

➤ 说明

定义解码通道参数结构体。

➤ 定义

```
typedef struct MI_VDEC_ChnParam_s
{
    MI_VDEC_DecodeMode_e eDecMode;
    MI_VDEC_OutputOrder_e eOutputOrder;
    MI_VDEC_VideoFormat_e eVideoFormat;
} MI_VDEC_ChnParam_t;
```

➤ 成员

成员名称	描述
eDecMode	解码模式，参考 MI_VDEC_DecodeMode_e 定义。默认为E_MI_VDEC_DECODE_MODE_ALL。
eOutputOrder	参考 MI_VDEC_OutputOrder_e 定义，默认为E_MI_VDEC_OUTPUT_ORDER_DISPLAY 按显示顺序输出数据帧。
eVideoFormat	参考 MI_VDEC_VideoFormat_e 定义。

※ 注意事项

- 暂不支持设置这些参数。

➤ 相关数据类型及接口

无。

3.17. MI_VDEC_VideoStream_t

➤ 说明

定义视频解码的码流结构体。

➤ 定义

```
typedef struct MI_VDEC_VideoStream_s
{
    MI_U8* pu8Addr;
    MI_U32 u32Len;
    MI_U64 u64PTS;
    MI_BOOL bEndOfFrame;
    MI_BOOL bEndOfStream;
} MI_VDEC_VideoStream_t;
```

➤ 成员

成员名称	描述
pu8Addr	码流包的地址。
u32Len	码流包的长度，以字节为单位。
u64PTS	码流包的时间戳，以毫秒为单位。
bEndOfFrame	当前帧是否结束，预留，当前只支持帧模式下按帧传送。

bEndOfStream	是否发完所有码流。当所有码流数据帧传送完毕时，bEndOfStream值需置为TRUE。
--------------	--

※ 注意事项

- 暂不支持按流发送。按帧传送时，每次需完整传送一帧数据。
- 码流帧数据附带PTS时，解码后输出数据输出相同PTS。当u64PTS = -2时，在解码后，VDEC会丢弃该帧。用户可根据需要进行设置。

- 相关数据类型及接口
无。

3.18. MI_VDEC_UserData_t

➤ 说明

定义用户数据结构体。

➤ 定义

```
typedef struct MI_VDEC_UserData_s
{
    MI_U8* pu8Addr;
    MI_U32 u32Len;
    MI_BOOL bValid;
} MI_VDEC_UserData_t;
```

➤ 成员

成员名称	描述
pu8Addr	用户数据的虚拟地址。
u32Len	用户数据的长度。以 byte 为单位。
bValid	当前数据的有效标识。取值范围：{TRUE, FALSE}。 TRUE：有效。 FALSE：无效。

※ 注意事项

- 目前暂不支持该参数相关功能。

- 相关数据类型及接口
无。

3.19. MI_VDEC_OutputPortAttr_t

➤ 说明

定义输出端口属性。

➤ 定义

```
typedef struct MI_VDEC_OutputPortAttr_s
```

```

{
    MI_U16 u16Width;           // Width of target image
    MI_U16 u16Height;          // Height of target image
}MI_VDEC_OutputPortAttr_t;

```

➤ 成员

成员名称	描述
u16Width	输出图像宽
u16Height	输出图像高

※ 注意事项

- 缩放宽和高需要按2对齐。
- 缩放范围[1/8, 1]。
- 不支持放大。
- 若裁剪和缩放同时开启，则裁剪宽和高不能小于缩放的宽和高。

➤ 相关数据类型及接口

无。

3.20. MI_VDEC_ErrHandlePolicy_t

➤ 说明

定义错误宏块处理策略结构体。

➤ 定义

```

typedef struct MI_VDEC_ErrHandlePolicy_s
{
    MI_BOOL bUseCusPolicy;      // FALSE: use default; TRUE: use customized value
    MI_U8 u8ErrMBPercentThreshold;
} MI_VDEC_ErrHandlePolicy_t;

```

➤ 成员

成员名称	描述
bUseCusPolicy	是否需要客制化控制错误帧输出策略。
u8ErrMBPercentThreshold	设置控制不输出数据帧时，错误宏块数占整张数据帧宏块数的比例值，取值范围[0, 100]。

※ 注意事项

- bUseCusPolicy 值为FALSE时代表不做客制化，u8ErrMBPercentThreshold为系统内置默认值30；bUseCusPolicy值为TRUE时，则使用客制化定义的u8ErrMBPercentThreshold数值。
- u8ErrMBPercentThreshold取值范围[0, 100]。例如数值为10时，代表错误宏块个数占整个数据帧宏块个数低于10%时，当前数据帧输出显示，否则(大于等于10%)不显示。其中数值为0时，代表只要有错误宏块都将不输出显示；数值为100时，代表不管有多少错误宏块当前数据帧都将会显示。
- u8ErrMBPercentThreshold数值是以宏块为基本单位计算比率值。

➤ 相关数据类型及接口

无。

3.21. MI_VDEC_CropCfg_t

➤ 说明

定义裁剪属性结构体。

➤ 定义

```
typedef struct MI_VDEC_CropCfg_s
{
    MI_BOOL bEnable; /* Crop region enable */
    MI_SYS_WindowRect_t stRect; /* Crop region */
} MI_VDEC_CropCfg_t;
```

➤ 成员

成员名称	描述
bEnable	开启/关闭裁剪功能。
stRect	裁剪属性。结构体MI_SYS_WindowRect_t参考mi_sys数据类型定义。

※ 注意事项

- 取值范围：u16X:[0, 4096-16], u16Y:[0, 4096-2], u16Width:[16, 4096], u16Height:[2, 4096];
- 对齐要求：u16X和u16Width需按16对齐，u16Y和u16Height需按2对齐。
- 若裁剪和缩放同时开启，则裁剪宽和高不能小于缩放的宽和高。

➤ 相关数据类型及接口

无。

4. 错误码

视频解码错误码如下表所示：

错误代码	宏定义	描述
0xA0082001	MI_ERR_VDEC_INVALID_DEVID	设备 ID 超出合法范围
0xA0082002	MI_ERR_VDEC_INVALID_CHNID	通道 ID 超出合法范围
0xA0082003	MI_ERR_VDEC_ILLEGAL_PARAM	非法参数或输入参数超过通道解码能力
0xA0082004	MI_ERR_VDEC_CHN_EXIST	试图创建已经存在的通道
0xA0082005	MI_ERR_VDEC_CHN_UNEXIST	通道不存在
0xA0082006	MI_ERR_VDEC_NULL_PTR	输入参数为空指针
0xA0082007	MI_ERR_VDEC_NOT_CONFIG	使用前未配置
0xA0082008	MI_ERR_VDEC_NOT_SUPPORT	该操作或者功能不支持
0xA0082009	MI_ERR_VDEC_NOT_PERM	非法操作
0xA008200C	MI_ERR_VDEC_NOMEM	分配内存失败，如系统内存不足
0xA008200D	MI_ERR_VDEC_NOBUF	分配缓存失败，如申请的数据缓冲区太大
0xA008200E	MI_ERR_VDEC_BUF_EMPTY	缓冲区中无数据
0xA008200F	MI_ERR_VDEC_BUF_FULL	缓冲区中数据满
0xA0082010	MI_ERR_VDEC_SYS_NOTREADY	系统没有初始化或者相关依赖的模块没有加载
0xA0082011	MI_ERR_VDEC_BADADDR	地址错误
0xA0082012	MI_ERR_VDEC_BUSY	系统忙
0xA0082013	MI_ERR_VDEC_CHN_NOT_START	通道未使能
0xA0082014	MI_ERR_VDEC_CHN_NOT_STOP	通道未禁用
0xA0082015	MI_ERR_VDEC_NOT_INIT	模块没有成功加载
0xA0082016	MI_ERR_VDEC_INITED	模块已成功加载
0xA0082017	MI_ERR_VDEC_NOT_ENABLE	解码器未使能
0xA0082018	MI_ERR_VDEC_NOT_DISABLE	解码器未禁用
0xA008201F	MI_ERR_VDEC_FAILED	失败

5. PROCFS 介绍

5.1. cat

➤ 调试信息

查看 vdec device 0:

```
# cat /proc/mi_modules/mi_vdec/mi_vdec0
```

查看 vdec device 1:

```
# cat /proc/mi_modules/mi_vdec/mi_vdec1
```

-----DEV Info-----										
DevID	UUID	MaxChnNum	TotEngCnt							
0	0	32	0							
-----CHN ATTR Info-----										
ChnID	CodecType	width	Height	BufSize	VideoMode	DpbBufMode	RefFrmNum			
0	H264	1920	1080	1048576	FRAME	INPLACE1	1			
1	H265	1920	1080	1048576	FRAME	INPLACE1	1			
2	H264	1080	1920	1048576	FRAME	INPLACE1	1			
3	H265	1080	1920	1048576	FRAME	INPLACE1	1			
-----CHN PARAM Info-----										
ChnID	bcusPolicy	ErrMBDropThrd	DisplayMode	bDisableLowLatency						
0	N	30	PREVIEW	0						
1	N	30	PREVIEW	0						
2	N	30	PREVIEW	0						
3	N	30	PREVIEW	0						
-----Crop & Scale Info-----										
ChnID	bcrop	cropX	cropY	cropW	cropH	bScale	ScaleW	ScaleH		
0	N	0	0	0	0	Y	960	540		
1	N	0	0	0	0	Y	960	540		
2	N	0	0	0	0	Y	960	540		
3	N	0	0	0	0	Y	960	540		
-----Decode Frame Info-----										
ChnID	DecW	DecH	DispW	DispH	RefRecFrmCnt	LinearFrmCnt				
0	1920	1088	960	540	2	0				
1	1920	1080	960	540	2	0				
2	1088	1920	960	540	2	0				
3	1080	1920	960	540	2	0				
-----Output Frame Info-----										
ChnID	btileMode	TileFormat	width	Height	Stride	PixelFmt				
0	N	NA	960	540	960	YUV420SP				
1	N	NA	960	540	960	YUV420SP				
2	N	NA	960	540	960	YUV420SP				
3	N	NA	960	540	960	YUV420SP				
-----CHN STATE-----										
ChnID	bstart	DecState	SendCnt	SendStrmSize	LeftStrmBytes	LeftCnt	DecFrmCnt	fps		
0	Y	5	124	2030919	0	0	121	25		
1	Y	5	126	981185	0	0	122	26		
2	Y	5	123	2627031	0	0	118	25		
3	Y	5	125	1282318	0	0	117	25		
ChnID	Start	Get	GetOK	Done	Drop	DropNovCL	Run	SeqChange		
0	123	123	123	121	0	2	230	0		
1	124	124	124	122	0	2	170	0		
2	121	121	121	118	0	3	192	0		
3	123	123	123	117	0	6	227	0		
ChnID	Issue	Complete	InitSeq	VlcInsuff	FrmDone	chkInitSeq	chkVlcInsuff	chkFrmDone	RcvRstCB	ChkRstCB
0	1	1	1	1	123	1	1	123	0	0
1	2	2	2	0	124	2	0	124	0	0
2	2	2	2	1	121	2	1	121	0	0
3	2	2	2	0	123	2	0	123	0	0
-----ISR STATE-----										
IsrCnt	InitSeq	VlcInsuff	Isr	STATE	FrmDone	UnderRun	HwTimeout	Other		
500	7	2			491	0	0	0		

➤ 调试信息分析

打印分为两部分，使用 **Private Vdec0 Info** 分隔开。上半部分为 **common** 信息，下半部分为 **vdec** 模块信息。主要记录了解码通道的使用情况及配置属性，可用于检查属性配置和当前通道的工作状态，便于 **debug**。

➤ 参数说明

参数		描述
DEV Info	DevID	硬件设备号。
	UUID	UUID 号。
	MaxChnNum	最大支持通道个数。
	TotEnqCnt	解码硬件队列排队帧数。
CHN ATTR Info	DevID	硬件设备号。
	ChnID	通道号。
	CodecType	解码协议类型 0: H264; 1: H265;
	Width	解码图像最大宽度。
	Height	解码图像最大高度。
	BufSize	VDEC 码流 buffer 大小，单位：byte。
	VideoMode	发送码流方式。 FRAME: 按帧发送。
	DpbBufMode	解码模式 INPLACE1: DBP_MODE_INPLACE_ONE_BUF; INPLACE2: DBP_MODE_INPLACE_TWO_BUF; NORMAL: DBP_MODE_NORMAL;
CHN PARAM Info	RefFrmNum	最大分配参考帧个数。
	DevID	硬件设备号。
	ChnID	通道号。
	bCusPolicy	是否由客户设置丢帧策略 Y: 客户设置; N: 客户不设置，使用默认;
	ErrMBDropThrd	丢帧策略生效时，错误宏块占整张数据帧的比例。
	DisplayMode	显示模式。 PREVIEW: 预览模式 PLAYBACK: 回放模式
Crop & Scale Info	bDisableLowLatency	通道是否支持 B 帧播放。 0: 不支持 B 帧播放。 1: 支持 B 帧播放。
	DevID	硬件设备号。

参数		描述
	ChnID	通道号。
	bCrop	是否设置裁剪功能。 N: 不设置。 Y: 设置。
	CropX	裁剪源图开始的 X 坐标。
	CropY	裁剪源图开始的 Y 坐标。
	CropW	裁剪源图的宽度。
	CropH	裁剪源图的高度。
	bScale	是否设置缩小功能。 N: 不设置。 Y: 设置。
	ScaleW	设置缩小输出的宽度。
	ScaleH	设置缩小输出的高度。
Decode Frame Info	DevID	硬件设备号。
	ChnID	通道号。
	DecW	源码流宽度。
	DecH	源码流高度。
	DispW	输出数据帧的宽度。
	DispH	输出数据帧的高度。
	RefRecFrmCnt	当前源码流解码所需要分配参考帧个数。
	linearFrmCnt	Output Buffer 分配的个数。
Output Frame Info	DevID	硬件设备号。
	ChnID	通道号。
	bTileMode	是否打开 tilemode 输出格式。 Y: 是。 N: 否。
	TileFormat	TileMode 开启后对齐的格式。 128x64: 128x64 字节对齐。 128x32: 128x32 字节对齐。 32x32: 32x32 字节对齐。 Invalid: 不支持。 NA: 没有开启 tilemode 输出格式。
	Width	输出数据帧的宽度。
	Height	输出数据帧的高度。
	Stride	输出数据帧需对齐后的宽度 stride。
	PixelFmt	输出数据帧的像素格式。 YUV420SP: NV12 数据帧格式输出。

参数		描述
CHN STATE	DevID	硬件设备号。
	ChnID	通道号。
	bStart	解码器是否启动。 Y: 启动, 进入解码状态。 N: 停止解码状态。
	DecState	当前通道解码状态。 0: 未定义。 1: 通道刚创建。 2: 等待 sps, pps 等关键数据帧。 3: 获取关键 sps, pps 信息完成状态。 4: 解码前申请参考帧等内存状态。 5: 申请到解码所需内存, 正常解码状态。 6: 通道关闭状态。
	SendCnt	应用层推流次数。
	SendStrmSize	应用层推流字节数。
	LeftStrmBytes	码流 buffer 中待解码的 byte 数。
	LeftCnt	相应于 SendCnt 参数, 待解码的次数。
	DecFrmCnt	成功解码输出的帧数。
	fps	当前解码帧率。
CHN STATE	DevID	硬件设备号。
	ChnID	通道号。
	Start	开始解码次数。
	Get	获取解码结果的次数。
	GetOK	成功获取到解码结果的次数。
	Done	成功解码并输出显示的次数。
	Drop	丢弃不输出显示的次数。
	DropNoVCL	丢弃无效数据帧的次数。
	Run	申请硬件解码次数。
	SeqChange	图像序列发生变化的次数。例如: SPS。
CHN STATE	DevID	硬件设备号。
	ChnID	通道号。
	Issue	请求解码器解析码流头信息的次数。
	Complete	解码器完成解析码流头信息的次数, 对应 Issue 请求。
	InitSeq	解码器完成解析码流头信息后, 对应 Issue 请求, 返回中断次数。

参数		描述
	VlcInsuff	VLC Buffer 不足的中断次数。
	FrmDone	解码器返回解码成功的中断的次数。
	ChkInitSeq	对应 InitSeq, 解码器返回完成解释码流头信息成功的中断后, 对应 VDEC 检查中断并处理的次数。
	ChkFrmDone	对应 FrmDone, 解码器返回解码成功的中断后, 对应 VDEC 检查中断并处理的次数。
	RcvRstCB	接收到解码器请求 Reset 的中断的次数。
	ChkRstCB	对应 RcvRstCB 中断请求, VDEC 处理解码器 Reset 请求的次数。
ISR STATE	IsrCnt	解码器上报中断的总次数。
	InitSeq	解码器上报解析码流头信息中断的总次数。
	VlcInsuff	解码器上报 VLC Buffer 不足的中断总次数。
	FrmDone	解码器上报解码完成的中断总次数。
	UnderRun	解码器上报任务线程还未解码完成的总次数。
	HwTimeout	解码器上报硬件解码超时的总次数。
	Other	解码器上报硬件其他中断总次数。

1.1. echo

vdec device 0:

/proc/mi_modules/mi_vdec/mi_vdec0

vdec device 1:

/proc/mi_modules/mi_vdec/mi_vdec1

功能	打开抓图或静帧功能的 debug log 相关信息到/proc/kmsg
命令	echo log [capture/refresh] [on/off] > /proc/mi_modules/mi_vdec/mi_vdec0
参数说明	capture/refresh: capture 抓图, refresh 静帧
	on/off: on 打开, off 关闭
举例	echo log refresh on > /proc/mi_modules/mi_vdec/mi_vdec0
	打开静帧功能的 debug log 相关信息到/proc/kmsg

功能	将指定通道输入的 bitstream 数据存储到指定路径
命令	echo dumpbs [chn] [path] > /proc/mi_modules/mi_vdec/mi_vdec0
参数说明	chn: 通道号 [0~63]
	path: dump 文件输出路径, 输入 off 代表停止存储码流。
举例	echo dumpbs 3 /mnt/dump/ > /proc/mi_modules/mi_vdec/mi_vdec0

	dump 通道 3 的应用推送到 bitstream buffer 的码流数据到/mnt/dump 目录下
--	---

功能	将指定通道的当前码流 buffer 的所有数据存储到指定路径。（整个码流 buffer 包含有效数据和无效数据，注意与前面 dumpbs 的区别）
命令	echo dumpbsb [chn] [path] > /proc/mi_modules/mi_vdec/mi_vdec0
参数说明	chn: 通道号 [0~63] path: dump 文件输出路径。
举例	echo dumpbsb 3 /mnt/dump/ > /proc/mi_modules/mi_vdec/mi_vdec0 dump 通道 3 的整个码流 buffer 的数据到/mnt/dump 目录下

功能	将指定通道解码后的输出数据帧存储指定路径
命令	echo dumpfb [chn] [path] [frmcnt] [bDumpAll] [bDetile] > /proc/mi_modules/mi_vdec/mi_vdec0
参数说明	chn: 通道号 [0~63] path: dump 文件输出路径 Frmcnt: dump 的帧数 bDumpAll: 0: 只存储 yuv buffer。 1: 同时存储 Y buffer, UV buffer 和 YUV buffer。 bDetile: 0: 不对将要存储的 YUV 做 detile (tilemode 模式下存储 tile 格式数据)。 1: 对将要存储的 YUV 做 detile (tilemode 模式下存储 YUV 格式数据)。
举例	通道 3, 存储路径为/mnt/dump/, 存储 99 帧, 不单独保存 Y buffer 和 UV buffer, 对将要存储的 YUV 做 detile echo dumpfb 3 /mnt/dump/ 99 0 1 > /proc/mi_modules/mi_vdec/mi_vdec0

功能	打开/关闭 flow checkpoint, 检查解码任务运行状态。（不建议客户使用，内部 debug）
命令	echo flowdbg [Status] > /proc/mi_modules/mi_vdec/mi_vdec0
参数说明	Status: on 打开 off 关闭
举例	echo flowdbg on > /proc/mi_modules/mi_vdec/mi_vdec0 打开 vdec flow checkpoint, 然后再敲 cat /proc/mi_modules/mi_vdec/mi_vdec0 可以在 vdec procfs 查看当前多个函数的 checkpoint

功能	打开/关闭丢帧开关, 丢弃所有的解码输出数据帧, 不输出到解码后端。
----	------------------------------------

命令	echo dropoutbuf [Status] > /proc/mi_modules/mi_vdec/mi_vdec0
参数说明	Status: on 打开 off 关闭
举例	echo dropoutbuf on > /proc/mi_modules/mi_vdec/mi_vdec0 丢弃将要输出的 frame buffer

功能	打开/关闭 vdec 性能统计开关。将输出更详细的 proc debug 信息。（不建议客户使用，内部 debug。）
命令	echo setperf [Status] > /proc/mi_modules/mi_vdec/mi_vdec0
参数说明	Status: on 打开 off 关闭
举例	echo setperf on > /proc/mi_modules/mi_vdec/mi_vdec0 打开 vdec 性能统计开关，然后再敲 cat /proc/mi_modules/mi_vdec/mi_vdec0 可以在 vdec procfs 查看当前解码耗时等数据。

功能	检查解码过程各状态消耗时间间隔。（不建议客户使用，内部 debug）
命令	echo flowstat [chn] [on/off] > /proc/mi_modules/mi_vdec/mi_vdec0
参数说明	chn: 通道号 [0~63] on/off: on 打开 off 关闭
举例	echo flowstat 3 on > /proc/mi_modules/mi_vdec/mi_vdec0 打开通道 3 的检查解码过程各状态消耗时间间隔功能。

功能	静帧功能内部 debug。（不开放客户使用，内部平台 debug。）
命令	echo refresh [chn] [pattern/dumpref/md5/clroutbuf] [on/off] [path] > /proc/mi_modules/mi_vdec/mi_vdec0
参数说明	chn: 通道号 [0~63] pattern/dumpref/md5/clroutbuf: pattern: 不使用解码器的参考帧，使用本地文件解码输出 NV12 数据帧。 dumpref: 存储解码器用于解码输出 NV12 的参考帧。 md5: 检查静帧整个流程所有的 md5 值是否一致，检查踩内存问题 debug 打开。 clroutbuf: 转码前清空 SCL 输出 buffer 的数据。 on/off: 打开与关闭。 path: 本地文件与存储文件的路径。
举例	echo refresh 3 md5 on /mnt/dump/ > /proc/mi_modules/mi_vdec/mi_vdec0 通道 3，打开静帧功能检查输出 md5 值。